WOJSKOWA AKADEMIA TECHNICZNA

im. Jarosława Dąbrowskiego

WYDZIAŁ CYBERNETYKI



PRACA DYPLOMOWA

STUDIA I°

Temat pracy: **PROJEKT I IMPLEMENTACJA PLATFORMY DO ROZGRYWEK TURNIEJÓW SILNIKÓW SZACHOWYCH**

INFORMATYKA

(kierunek studiów)

MOBILNE SYSTEMY KOMPUTEROWE

(specjalność)

Dyplomant:

Hubert GODAWA

Promotor:

dr inż. Adrian WOŹNIAK

Warszawa 2025

OŚWIADCZENIE

Wyrażam zgodę / ~~nie wyrażam zgody~~*

na udostępnianie mojej pracy w czytelni Archiwum WAT.

Dnia 07.01.2025 Hubert Godawa

(podpis)

**Niepotrzebne skreślić*

Spis treści

Rozdział I. Zakres funkcjonalny 5
I.1. Opis przypadku użycia „Testuj komendy silnika szachowego” 6
I.2. Opis przypadku użycia „Aktywuj silnik szachowy na stronie” 7
I.3. Opis przypadku użycia „Rozegraj grę” 9
I.4. Opis przypadku użycia „Rozegraj turniej” 11
Rozdział II. Architektura rozwiązania 13
II.1. Architektura logiczna 13
II.2. Architektura technologiczna 14
II.3. Architektura kodu 14
II.4. Opis bazy danych 16
II.5. Mechanizmy bezpieczeństwa 19
II.6. Charakterystyka środowiska wytwórczego i uruchomieniowego 20
Rozdział III. Opis implementacji 22
III.1. Interfejs użytkownika 22
III.2. Szachowe algorytmy walidacji ruchu 29
III.3. Szachowe algorytmy rozstrzygania rezultatu ruchu 39
III.4. Klasa zarządzająca przebiegiem gry szachowej 49
III.5. Klasa zarządzająca przebiegiem turnieju szachowego 51
III.6. Przykładowa implementacja serwera z silnikiem szachowym 54
III.7. Testy zaimplementowanych funkcji systemu 56
III.8. Kierunki rozwoju systemu 57
Podsumowanie 59
Bibliografia 60
Spis rysunków 61
Spis kodów 62
Załączniki 63

Wstęp

Szachy od wieków stanowią symbol strategicznego myślenia i rywalizacji umysłów. Ich historia sięga setek lat, jednak przełom w rozwoju tej gry nastąpił w 1997 roku, kiedy pierwszy raz w ich historii komputer szachowy pokonał ludzkiego mistrza świata. Był to moment, który zapoczątkował nową erę – postęp technologiczny sprawił, że silniki szachowe stały się narzędziem nie tylko dla profesjonalistów, ale także dla entuzjastów zainteresowanych analizą gier i tworzeniem własnych algorytmów. Dzięki postępowi technologicznemu silniki szachowe stały się bardziej dostępne, kompaktowe i wszechstronne, znajdując zastosowanie nie tylko w analizie partii, ale też w autonomicznych rozgrywkach między komputerami.

W ramach niniejszej pracy wytworzono platformę internetową, udostępniającą przyjazne środowisko dla rozwoju amatorskich silników szachowych, które mogą stanowić doskonałą okazję do nauki programowania, a także zgłębiania zagadnień z obszaru teorii gier oraz algorytmów i struktur danych. Opracowany projekt umożliwia organizację gier i turniejów szachowych, nie tylko dla graczy komputerowych, ale też ludzkich. Dzięki takiemu podejściu, utworzone rozwiązanie dostarcza kompletne doświadczenie szachowe - zarówno osobom zainteresowanym poznawaniem tajników tej gry, jak i tym którzy chcą na własnych oczach przekonać się o możliwościach współczesnych komputerów szachowych oraz samemu dokonać prac nad ich rozwojem.

Na platformie zaimplementowano autorskie algorytmy szachowe, które dzięki rozbiciu na elementarne części są łatwe do zrozumienia i utrzymania. Prostota opracowanych algorytmów sprzyja dalszej implementacji wariantów szachowych, której mogą się podjąć osoby zainteresowane nauką programowania i algorytmiki bez konieczności wprowadzania znaczących zmian w pozostałych częściach systemu.

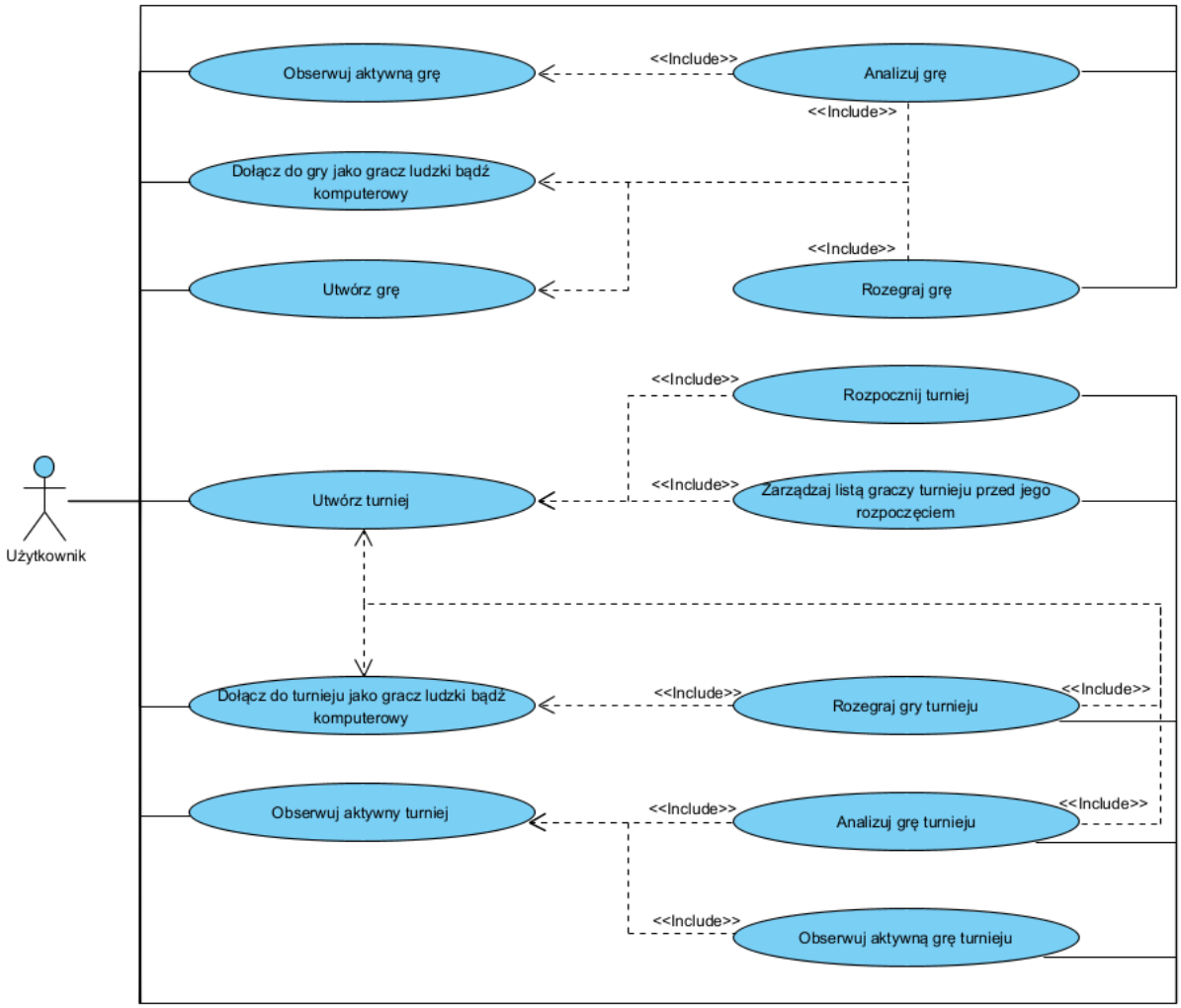
Treść pracy podzielono na trzy rozdziały. W pierwszym z nich zaprezentowano funkcje dostarczane przez wytworzony system, z wyróżnieniem najciekawszych i najbardziej charakterystycznych przypadków użycia na platformie. W tej części pracy prezentowany jest przebieg gier i turniejów, a także proces aktywacji oraz testowania silnika szachowego na stronie.

Rozdział drugi przedstawia analizę architektury opracowanego systemu - od najbardziej abstrakcyjnego poziomu po detale praktyczne. W tym fragmencie pracy wspomniano też o różnych technicznych aspektach projektu, takich jak wykorzystane mechanizmy bezpieczeństwa i narzędzia wytwórcze.

W rozdziale trzecim zaprezentowano interfejsy użytkownika, a także omówiono informatyczne rozwiązania zaprojektowane w ramach systemu. Wyjaśniono takie aspekty pracy jak konstrukcja i działanie opracowanych w systemie algorytmów szachowych, implementacja przykładowego serwera z silnikiem szachowym jakiego użytkownik mógłby użyć na stronie oraz budowa klas odpowiedzialnych za poprawny przebieg gier i turniejów w systemie. Na koniec tego rozdziału przedstawiono testy zaimplementowanych funkcji oraz potencjalne kierunki rozwoju systemu.

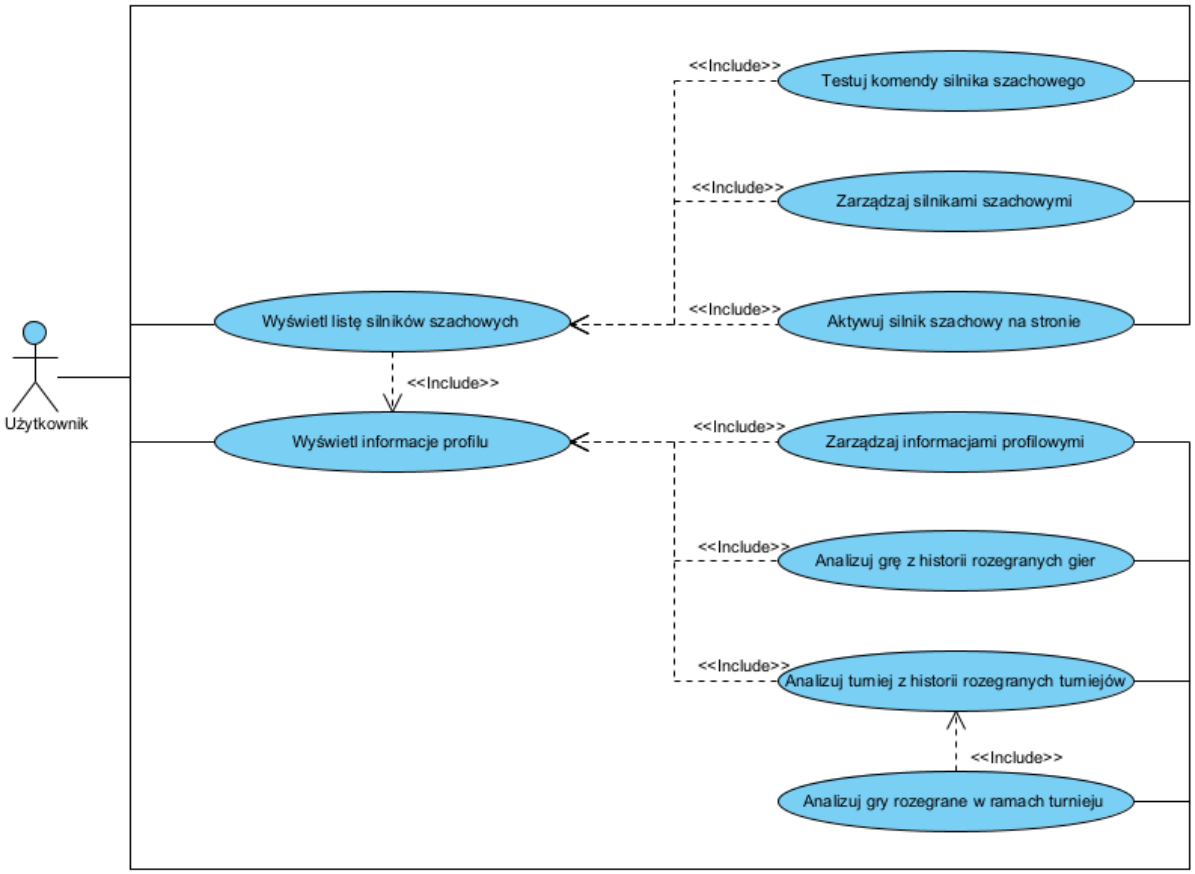
Rozdział I. Zakres funkcjonalny

Funkcje systemu zostały zaprojektowane z myślą o dostarczeniu użytkownikowi końcowemu przestrzeni do rozwoju i testowania silników szachowych, jednakże zostały rozwinięte o możliwość poszerzania umiejętności i wiedzy szachowej bez ich udziału. W związku z powyższym platforma oferuje rozgrywanie gier i turniejów nie tylko pomiędzy silnikami szachowymi, ale również między graczami ludzkimi oraz w trybie mieszanym (człowiek-komputer). Ze względu na wspomniane możliwości przy implementacji projektu poświęcono uwagę nie tylko obszarowi przeprowadzania rozgrywek, ale też obszarowi zarządzania profilem użytkownika oraz silnikami szachowymi udostępnianymi przez użytkowników na stronie. Funkcje tych dwóch obszarów zostały zaprezentowane na diagramach przypadków użycia (Rys. 1, Rys. 2).



Rys. 1. Diagram przypadków użycia panelu gier użytkownika

Źródło: Opracowanie własne



Rys. 2. Diagram przypadków użycia panelu profilu użytkownika

Źródło: Opracowanie własne

W obszarze przeprowadzania rozgrywek warto zaznaczyć, że tworzenie gier i turniejów odbywa się z możliwością wyboru ich formatu czasowego, trybu graczy, a także wymagań jakie muszą spełniać osoby dołączające. To jakim kolorem figur użytkownik tworzący grę będzie rozgrywać partię jest decydowane losowo przez system. Krótkiego wyjaśnienia mogą też wymagać przypadki użycia „Analizuj grę" oraz „Analizuj grę turnieju", ich funkcje polegają na możliwości korzystania z interfejsów pozwalających na przeglądanie wszystkich pozycji oraz ruchów, które zaszły w ramach wybranej rozgrywki.

Analiza gier i turniejów w obszarze zarządzania profilem użytkownika odbywa się na tej samej zasadzie co w obszarze rozgrywek. Warto dodać, że przy analizowaniu turnieju użytkownik ma wgląd nie tylko w wyniki i przebieg wszystkich gier rozegranych w jego ramach, ale także w punktację jego uczestników.

I.1. Opis przypadku użycia „Testuj komendy silnika szachowego"

Pierwszy przypadek użycia, któremu warto przypisać szczególną uwagę to „Testuj komendy silnika szachowego". Funkcja ta jest dostępna dla użytkownika dopiero gdy poda on na stronie adres URL swojego serwera z silnikiem szachowym i wykorzystywana jest poprzez interfejs chatu z silnikiem znajdujący się

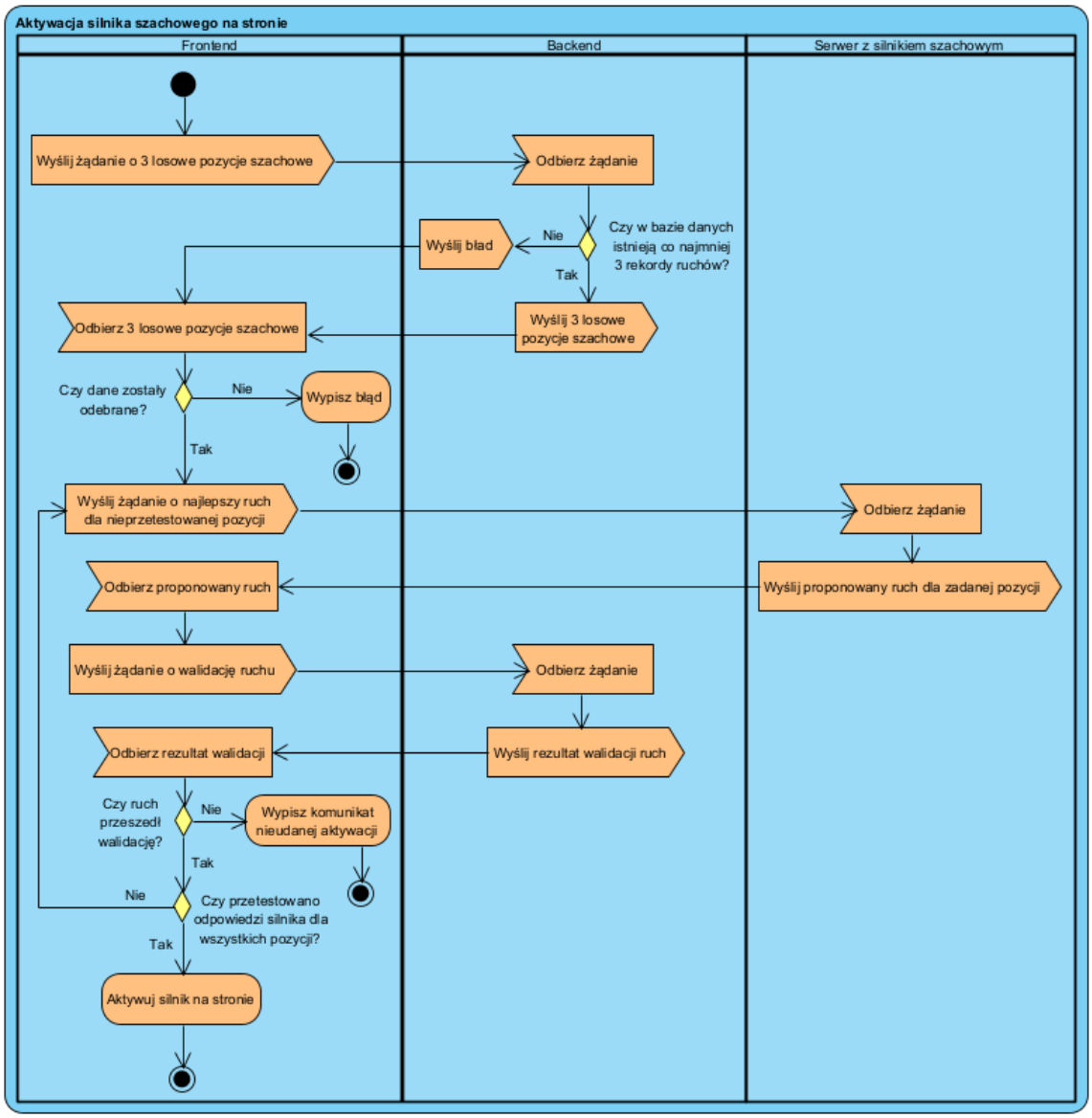
w sekcji zarządzania silnikami szachowymi. Korzystanie z tej funkcji polega na wysyłaniu komend w polu chatu, których obsługę użytkownik zaimplementował na swoim serwerze. W odpowiedzi wysyłane i wyświetlane są dowolne dane, które zwróci serwer użytkownika.

Funkcja testowania komend silnika szachowego choć prosta, jest kluczowa dla zbadania poprawności działania serwera użytkownika. W celu nawiązania poprawnej komunikacji z platformą, a także zaimplementowania odpowiednich punktów końcowych (ang. *endpoint*) API (ang. *Application Programming Interface*) przez użytkownika na swoim serwerze, na platformie utworzono specjalną stronę informacyjną, wprowadzającą programistów korzystających ze strony w wymagania jakie udostępniany serwer powinien spełniać. W ramach tej strony udostępniono również link do repozytorium zewnętrznego z przykładową implementacją serwera z silnikiem szachowym, która została wykonana w ramach projektu.

W implementacji serwera użytkownika wymagana jest głównie obsługa komend zdefiniowanych w ramach protokołu UCI (ang. *Universal Chess Interface*), czyli najpopularniejszego na świecie zbioru reguł komunikacji silnika szachowego z interfejsami użytkownika. W skład protokołu UCI wchodzą komendy pozwalające na potwierdzenie gotowości silnika, ustawienie rozpatrywanej pozycji, uzyskanie najlepszego ruchu dla pozycji i wiele więcej. Żeby poprawnie obsługiwać przypadek użycia „Testuj komendy silnika szachowego” wymaga się od użytkownika przede wszystkim zaimplementowania na jego serwerze punktu końcowego API ‘/chat’, który przy otrzymywaniu żądań z platformy powinien wywoływać wskazane w żądaniu komendy UCI zaimplementowane w silniku i zwracać odpowiedzi, jeśli występują.

I.2. Opis przypadku użycia „Aktywuj silnik szachowy na stronie”

Aktywacja silnika szachowego na stronie to bardzo ważna funkcja systemu zapewniająca integralność i poprawność działania silnika przed jego wykorzystaniem w grach i turniejach, więc jest ona wymagana przed dołączeniem z jego udziałem do rozgrywek. Przypadek użycia polega na przetestowaniu działania silnika poprzez automatyczne wysłanie trzech losowo pobranych z bazy danych pozycji szachowych, dla których silnik musi zaproponować poprawne ruchy. Proces aktywacji silnika jest realizowany za każdym wejściem na stronę, aby upewnić się, że od ostatniej wizyty silnik nie uległ awarii lub niepoprawnej modyfikacji. Dzięki takiemu podejściu system zachowuje niezawodność i kontroluje jakość działania silników szachowych pomiędzy sesjami użytkownika. Poprawne przejście testów stanowi także ważną dla użytkownika informację o zgodności jego silnika szachowego z wymaganiami systemu oraz powszechnie uznawanymi regułami protokołu UCI. Proces aktywacji przedstawiono dokładnie na diagramie aktywności (Rys. 3).



Rys. 3. Diagram aktywności procesu aktywacji silnika szachowego na stronie

Źródło: Opracowanie własne

Warto zwrócić uwagę, że diagram ze względu na poziom złożoności procesu został uproszczony i nie bierze pod uwagę implementacji takich mechanizmów jak na przykład zwracanie błędu w przypadku braku odpowiedzi od silnika. Sama komunikacja z silnikiem szachowym również została przedstawiona w sposób uproszczony, w rzeczywistości przed wysłaniem żądania o najlepszą pozycję sprawdzana jest gotowość silnika oraz ustawiana jest w nim rozpatrywana pozycja. Należy zaznaczyć, że proces aktywacji silnika szachowego działa tylko wtedy, gdy w bazie danych istnieje chociaż jedna rozegrana gra składająca się z minimum trzech ruchów. Z tego powodu w przypadku udostępniania projektu osobom trzecim, jeśli baza danych jest pusta należy rozegrać jedną grę między graczami ludzkimi przed dodawaniem silników na stronie.

I.3. Opis przypadku użycia „Rozegraj grę”

Przypadek użycia „Rozegraj grę” stanowi najważniejszy element systemu, umożliwiający pełnowartościową rozgrywkę szachową zarówno pomiędzy graczami ludzkimi jak i komputerowymi. Proces rozgrywki jest złożony z kilku etapów, które rozpoznawane są przez system na podstawie dwóch pól rekordu gry w bazie danych, określających czas rozpoczęcia i zakończenia partii: ‘start_time’ oraz ‘end_time’. Pola te stanowią głównie dane statystyczne, pozwalające użytkownikowi lepiej zarządzać historią rozegranych gier, jednak na podstawie tego czy posiadają wartość, można dynamicznie wyróżnić trzy stany przedstawione w tabeli (Tab. 1).

Tab. 1. Stany gry szachowej

Lp.	Wartość pola start_time	Wartość pola end_time	Stan rozgrywki
1	Brak wartości	Brak wartości	Czeka na rozpoczęcie
2	Czas rozpoczęcia	Brak wartości	W trakcie rozgrywania
3	Czas rozpoczęcia	Czas zakończenia	Zakończona

Źródło: Opracowanie własne

I.3.1. Etap oczekiwania gry na rozpoczęcie

Pierwszy stan gry, czyli oczekiwanie na rozpoczęcie zachodzi po utworzeniu gry szachowej przez użytkownika. W tym etapie gra wyświetla się na liście gier oczekujących na przeciwnika i każdy zalogowany użytkownik spełniający wymagania ustalone przez twórcę może się do niej dołączyć. Po dołączeniu się przeciwnika gra automatycznie przechodzi w stan aktywnej rozgrywki. Jeśli nikt się nie dołączy i twórca gry postanowi opuścić rozgrywkę przed jej rozpoczęciem, to jest ona automatycznie usuwana z bazy danych przez system.

I.3.2. Etap rozgrywania partii szachowej

W trakcie gry zawodnicy na przemian wykonują ruchy na planszy szachowej, które każdorazowo poddawane są walidacji przy użyciu autorskich algorytmów szachowych zaimplementowanych w systemie. W przypadku gdy ruch wykonywany jest przez gracza ludzkiego, otrzymuje on audiowizualne powiadomienie o zgodności jego ruchu z zasadami szachów. Ruchy uznane przez system za nieprawidłowe wymagają powtórzenia. Gracze komputerowi (silniki szachowe) dla każdego ruchu dostają w sumie trzy okazje wykonania poprawnego zagrania. Zadaniem programisty udostępniającego serwer z silnikiem na stronie, jest zaimplementowanie mechanizmu podającego inny ruch przy każdej próbie. Dodatkowo, czas dozwolony na wykonanie ruchu przez silnik jest dynamicznie określany przez system, w celu zapobiegania sytuacji nadmiernego przeciągania rozgrywki w czasie przez wadliwy silnik. Przy

trzech nieudanych próbach wykonania ruchu przez silnik, gra automatycznie jest uznawana za przegraną przez jego właściciela.

W ramach rozgrywki szachowej zaimplementowano liczne dodatkowe funkcje wzbogacające doświadczenia graczy, przy użyciu mechanizmów dynamicznej komunikacji. Najważniejszą z nich są zegary szachowe, zarządzane automatycznie przez system. Zaczynają one odliczanie dla gracza białego automatycznie przy rozpoczęciu gry i przełączają się po wykonywaniu ruchów przez graczy. W przypadku upłynięcia czasu na zegarze u jednego z zawodników, automatycznie ogłaszana jest jego przegrana. Gracze za obopólną zgodą mogą też cofnąć ostatni zagrany ruch na planszy, uzgodnić remis, a także konwersować na chacie w dowolnym momencie rozgrywki. W celu uzyskania wysokiej interaktywności, wszystkie zmiany stanu gry oraz znaczące akcje również wypisywane są przez system na chacie.

I.3.3. Etap zakańczania rozgrywki

Przy stwierdzeniu zakończenia gry, oprócz czasu zakończenia zapisywany jest rezultat, w którego skład wchodzi wynik partii, rodzaj jej zakończenia oraz przyrost/spadek rankingu graczy. W tabeli (Tab. 2) przedstawiono rodzaje zakończenia gry jakie mogą wystąpić na stronie. Punktacja rankingowa graczy odbywa się w systemie ELO, opracowanym przez amerykańskiego fizyka Arpada Elo, biorącego pod uwagę takie czynniki jak obecny ranking, liczba rozegranych gier oraz wynik partii.

Tab. 2. Rodzaje zakończenia gry szachowej

Lp.	Rodzaj zakończenia gry	Przyczyna wystąpienia
1	Wygrana przez szach-mat	Gracz zmatował wrogiego króla
2	Wygrana przez rezygnację	Przeciwnik kliknął guzik rezygnacji na stronie
3	Wygrana przez rozłączenie	Przeciwnik stracił połączenie ze stroną gry
4	Wygrana przez błąd silnika	Silnik szachowy przeciwnika trzy razy z rzędu zaproponował niedozwolony ruch
5	Wygrana przez czas	Czas przeciwnika na jego zegarze upłynął
6	Remis przez pat	Gracz, którego jest tura nie może wykonać żadnego dozwolonego ruchu
7	Remis przez brak materiału	Żaden z graczy nie posiada figur umożliwiających zamatowanie króla przeciwnika
8	Remis przez trzykrotne powtórzenie pozycji	Pozycja na planszy powtórzyła się trzy razy
9	Remis przez zasadę 50 ruchów	Żaden z graczy nie wykonał bicia ani ruchu pionkiem przez 50 kolejnych ruchów
10	Remis przez obopólną zgodę	Obaj gracze zgodzili się na remis poprzez guzik na stronie

Źródło: Opracowanie własne

I.4. Opis przypadku użycia „Rozegraj turniej”

Przypadek użycia „Rozegraj turniej” jest ostatnim wymagającym szczegółowego opisu. Rozgrywanie turnieju odbywa się w trzech fazach analogicznych do stanów zaprezentowanych w przypadku użycia „Rozegraj grę” (patrz Tab. 1). Można więc wyróżnić stany oczekiwania na rozpoczęcie, aktywnego rozgrywania oraz zakończenia turnieju.

I.4.1. Etap oczekiwania na rozpoczęcie turnieju

W stanie oczekiwania na rozpoczęcie turnieju, jego twórca ma kontrolę nad listą zawodników i jest w stanie modyfikować dostępna liczbę miejsc dla graczy oraz usuwać zapisanych uczestników. Co najważniejsze, w przypadku użycia „Rozegraj turniej” o rozpoczęciu rozgrywek nie decyduje automatycznie system, tylko jest to również akcja wykonywana przez jego twórcę. Warto zaznaczyć, że jeśli na tym etapie twórca turnieju opuści stronę to system automatycznie przypisze jego rolę pierwszemu użytkownikowi zapisanemu na liście zawodników, co zapewnia ciągłość zarządzania. W przeciwieństwie do gry szachowej, oczekiwanie na rozpoczęcie turnieju nie musi być w pełni pasywne, gdyż już na tym etapie można rozmawiać z innymi oczekującymi zawodnikami poprzez chat. Podobnie jak przy rozgrywaniu gry, jeśli przed rozpoczęciem turnieju wyjdzie ostatni zawodnik na liście, turniej jest automatycznie usuwany z bazy danych.

I.4.2. Etap rozgrywania turnieju szachowego

Po rozpoczęciu turnieju gracze automatycznie dobierani są w pary przez logikę systemu i przekierowywani do ekranu gier, gdzie rozgrywają partię aż do jej zakończenia. W przypadku nieparzystej liczby zawodników, w każdym momencie trwania turnieju jeden z graczy oczekuje na zakończenie innych partii. Turniej odbywa się w trybie kołowym, więc jego zakończenie następuje dopiero w momencie, gdy każdy zawodnik rozegra partię ze wszystkimi pozostałymi graczami.

Przy implementacji systemu przeprowadzania turniejów zdecydowano się na dopuszczanie jedynie gier w formacie człowiek-człowiek oraz komputer-komputer. Rozgrywki mieszane między graczami ludzkimi, a komputerowymi są niedozwolone, gdyż w celu uczciwego przeprowadzenia turnieju wymagałyby one dwukrotnie zwiększonej liczby gier, ponieważ obaj gracze z każdej pary musieliby zagrać z silnikiem szachowym przeciwnika. Takie rozwiązanie byłoby nie tylko wymagające w implementacji, ale mogłoby także zbytnio komplikować tabele rezultatów turnieju. Dodatkowo, podczas gdy gra człowieka z silnikiem szachowym jest ciekawym i pouczającym eksperymentem, to jej charakter nie wpisuje się w rankingową naturę turnieju, gdyż zakładając nawet średni poziom zaawansowania silnika szachowego, realistycznie prawie zawsze wygra on z graczem ludzkim.

W trakcie rozgrywania turnieju, jeśli jeden z zawodników go opuści, u wszystkich graczy automatycznie usuwane są rezultaty gier z tym graczem. Takie

podejście zapewnia sprawiedliwy przebieg turnieju, gdzie każdy z graczy rozegra w jego ramach równą ilość gier. Jeśli w trakcie trwania turnieju wyjdą wszyscy zawodnicy, to podobnie jak przed jego rozpoczęciem zostanie on usunięty z bazy danych.

Na koniec warto zauważyć, że w trakcie trwania turnieju, zawodnicy oczekujący na przydzielenie do gry oraz obserwatorzy mogą swobodnie oglądać rozgrywki w toku, rezultaty gier i punktacje końcowe graczy.

I.4.3. Etap zakańczania turnieju

W momencie, gdy system wykryje, że wszystkie gry turnieju zostały rozegrane, wybierany jest zwycięzca i wynik turnieju zapisywany jest w bazie danych. W przypadku gdy wśród najwyżej punktowanych graczy jest więcej niż jedna osoba, wynik jest dodatkowo rozstrzygany poprzez zastosowanie kolejnych metod przełamywania remisu, aż do momentu wyłonienia zwycięzcy. Metody te to kolejno:

- Metoda Sonneborn-Bergera,
- Metoda wybierania osoby z największą liczbą zwycięstw,
- Metoda wybierania osoby z największym rankingiem ELO,
- Metoda losowania zwycięzcy.

W celu transparentności proces rozstrzygania zwycięzcy jest jawnie prezentowany przez system poprzez interfejs chatu użytkowników, podobnie jak inne wydarzenia i zmiany stanu turnieju.

Rozdział II. Architektura rozwiązania

II.1. Architektura logiczna

Architektura logiczna opracowanego systemu opiera się na podejściu trójwarstwowym, w ramach którego można wyodrębnić następujące warstwy: prezentacji, logiki biznesowej oraz dostępu do danych. Taki układ jest jednym z najpowszechniej stosowanych wzorców architektonicznych w inżynierii oprogramowania i dla każdego z elementów definiuje ściśle określony zakres odpowiedzialności:

- Warstwa prezentacji (Frontend): Warstwa ta odpowiada za interakcję z użytkownikiem. Posiada ona interfejs graficzny oraz prostą logikę, która zazwyczaj służy do kontrolowania danych wprowadzanych i wyświetlanych użytkownikowi.
- Warstwa logiki biznesowej (Backend): Warstwa ta jest przede wszystkim odpowiedzialna za obsługę żądań przesyłanych z poziomu warstwy prezentacji i działa jako mediator pomiędzy frontendem i bazą danych. Backend oprócz przekazywania danych, odpowiedzialny jest za realizację logiki biznesowej, implementuje zatem reguły biznesowe i procesy definiujące sposób działania aplikacji. Warstwa logiki biznesowej zazwyczaj odpowiedzialna jest też za kluczowe mechanizmy bezpieczeństwa – szyfrowanie danych, autoryzację użytkowników i przetwarzanie danych wrażliwych.
- Warstwa dostępu do danych (Baza danych): Odpowiada za trwałe przechowywanie informacji oraz operacje na danych, takie jak ich zapis, odczyt i aktualizacja. W tej warstwie wykorzystywana jest relacyjna baza danych, która zapewnia strukturalne przechowywanie danych, a także ich wysoką integralność i spójność.

W projekcie pomiędzy frontendem i backendem, zdecydowano się dodatkowo zastosować mechanizm komunikacji wykorzystujący gniazda sieciowe (ang. *websocket*) TCP, pozwalający na dynamiczne aktualizacje i powiadomienia na stronie. Jest on kluczowy dla szybko zmieniających się elementów strony, takich jak zegary szachowe.

Pewną rolę odgrywa baza danych NoSQL, służąca do tymczasowego przechowywania identyfikatorów użytkowników podczas komunikacji za pomocą gniazd sieciowych. Dzięki temu mechanizmowi, backend strony jest w stanie rozróżniać odświeżenie strony przez użytkownika od jego całkowitego rozłączenia.

II.2. Architektura technologiczna

Architektura technologiczna systemu opiera się na zestawie nowoczesnych narzędzi, powszechnie stosowanych w środowiskach wytwarzania stron webowych. Architekturę technologiczną najłatwiej opisać opierając się na wcześniej przedstawionej logicznej architekturze trójwarstwowej:

- Frontend: Warstwa prezentacji została zaimplementowana w technologii Angular opracowanej przez firmę Google. Technologia ta pozwala na tworzenie aplikacji typu SPA (ang. *Single Page Application*), w których całość logiki frontendowej jest ładowana jednorazowo, a dalsza komunikacja z backendem odbywa się asynchronicznie przez REST API lub gniazda sieciowe. REST API, oparte na standardach HTTP, umożliwia prostą realizację operacji CRUD (ang. Create, Read, Update, Delete) na zasobach aplikacji. Dzięki temu zmieniają się tylko dane, a nie cała strona, co minimalizuje czas przełączania widoków.
- Backend: Logika biznesowa systemu została zrealizowana w technologii Django, który jest wysokopoziomowym frameworkiem opartym na języku Python. W projekcie rozszerzono standardowe funkcje Django o paczkę ‘Django Channels’ co pozwoliło na łatwą i intuicyjną implementację oraz obsługę gniazd sieciowych TCP. Wykorzystano też rozszerzenia umożliwiające integrację z bazą danych NoSQL oraz mechanizmy autentykacji oparte na tokenach JWT (ang. *JSON Web Token*). W backendzie zastosowano także DRF (ang. *Django Rest Framework*) w celu zaimplementowania REST API, umożliwiającego efektywną komunikację między frontendem i backendem.
- Bazy danych: System korzysta z relacyjnej bazy danych PostgreSQL, która zapewnia wysoką niezawodność i zgodność ze standardami SQL, a także umożliwia przechowywanie złożonych struktur danych w formacie JSON. Baza danych NoSQL została zrealizowana w Redis, który charakteryzuje się wysoką wydajnością w operacjach wykonywanych w pamięci.

II.3. Architektura kodu

Architekturę kodu zorganizowano kierując się powszechnie stosowanymi standardami w wykorzystywanych technologiach, bazujących na popularnych wzorcach projektowych z obszaru rozwijania stron internetowych.

II.3.1. Architektura kodu warstwy prezentacji

Architektura kodu Angulara, w którym wykonano frontend aplikacji nie posiada oficjalnej nazwy, ale często określa się ją jako podejście oparte na komponentach (ang. *component-based architecture*). Kluczowe elementy tej architektury, zaimplementowane w systemie na podstawie pracy autorów [1] i dokumentacji [2], to:

- Komponenty (ang. *Components*) – są podstawowymi elementami budowy aplikacji Angular. Każdy komponent odpowiada za zarządzanie konkretną częścią interfejsu użytkownika i jego logiką. Komponenty składają z trzech głównych elementów: pliku TypeScript sterującego logiką komponentu, szablonu HTML definiującego strukturę interfejsu użytkownika oraz styli CSS opisujących wygląd i zachowanie struktury HTML.
- Serwisy (ang. *Services*) – służą do przechowywania i zarządzania logiką aplikacji oraz do komunikacji z backendem. Dzięki serwisom komponenty są odciążone od zadań związanych z obsługą danych, co poprawia ich czytelność i strukturyzację projektu. Serwisy programuje się całkowicie w języku TypeScript.
- Interfejsy (ang. *Interfaces*) – w Angular służą głównie do definiowanych struktur danych stosowanych w warstwie frontendu. Korzystanie z nich nie jest zawsze koniecznie, lecz zalecane dla zachowania czytelności projektu. Interfejsy podobnie jak inne elementy programowane są w TypeScript.

Angular oprócz wspomnianych elementów architektury za pomocą odpowiednio dedykowanych plików w prosty sposób definiuje takie aspekty aplikacji jak: nawigacja między widokami (ang. *App-Routing*), ochrona tras określająca dostępność widoków dla użytkownika (ang. *Auth-Guard*), czy spis komponentów oraz rozszerzeń stosowanych w projekcie (ang. *App-Module*).

II.3.2. Architektura kodu warstwy logiki biznesowej

Django oparty jest na architekturze MVT (ang. *Model-View-Template*), polegającej na rozdzieleniu odpowiedzialności plików na: opis struktur danych, logikę biznesową oraz prezentację danych użytkownikowi. W przypadku gdy Django używany jest wyłącznie w implementacji warstwy biznesowej projektu (tak jak w omawianym systemie), pomijana jest część prezentacyjna (ang. *Template*) wzorca MVT. Na podstawie pracy autorów [3], oficjalnej dokumentacji Django [4], a także dokumentacji Django REST [5] wykorzystano następujące elementy w języku Python:

- Modele (ang. *Models*) – odpowiadają za definicję struktur danych przechowywanych w bazie danych. Na ich podstawie generowane są migracje, czyli specjalne pliki umożliwiające synchronizację struktury bazy danych z modelami aplikacji. Migracje automatyzują proces tworzenia tabel w bazie danych, a także modyfikowania ich kolumn i relacji, co pozwala na łatwe zarządzanie zmianami w bazie danych w trakcie rozwoju projektu.
- Widoki (ang. *Views*) – przetwarzają żądania z frontendu i w większości wypadków wykonują odpowiednie operacje na bazie danych, a następnie zwracają odpowiedzi. W projekcie rozdzielono widoki tematycznie na osobne pliki, w celu poprawienia struktury projektu przy ich dużej ilości.

- Serializery (ang. *Serializers*) – konwertują obiekty Django na format JSON, który wykorzystywany jest w ramach komunikacji backendu z frontendem. Dzięki zdefiniowaniu serializerów dane można w łatwy sposób przesyłać do klienta i walidować ich poprawność przy odbiorze z warstwy prezentacji.
- Ścieżki – Plik ze ścieżkami (ang. *Urls*) stanowi spis wszystkich punktów końcowych udostępnianych przez backend, a także ich mapowanie na widoki Django.

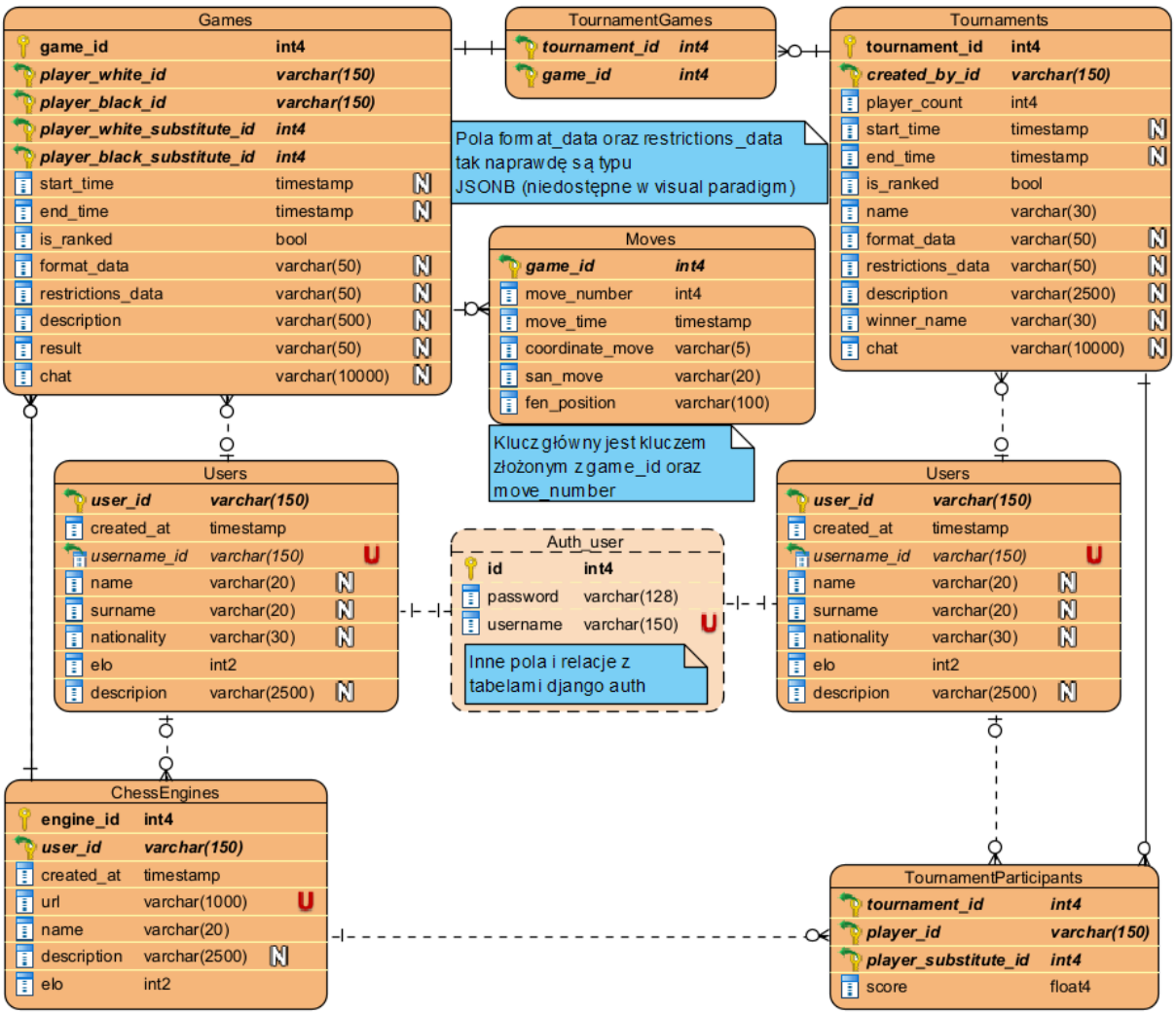
Oprócz opisanych elementów, które stanowią podstawę większości projektów Django, w projekcie zastosowano kilka nieco rzadszych rozwiązań zawartych w następujących fragmentach aplikacji:

- Konsumenci (ang. *Consumers*) – wykorzystywani są w Django do obsługi komunikacji poprzez gniazda sieciowe. W omawianym projekcie plik ‘Konsumenci’ umożliwia dynamiczną komunikację między graczami oraz aktualizacje stanu gry szachowej i turniejów.
- Menadżerzy (ang. *Managers*) – plik menadżerzy jest oryginalną częścią backendu, w której zdefiniowano klasy zarządzające przebiegiem gier i turniejów. Współpracują one ściśle z wiadomości odbieranymi w pliku ‘Konsumenci’. To właśnie na podstawie tych wiadomości inicjalizowane są obiekty tych klas, które na podstawie następujących wiadomości aktualizują stan rozgrywek i decydują o ich zakończeniu.
- Algorytmy szachowe – bardzo ważną częścią warstwy logiki biznesowej są autorskie algorytmy szachowe, wymyślone i napisane jeszcze przed pomysłem utworzenia platformy szachowej, w ramach gry komputerowej „Space Engineers”. Oryginalne algorytmy posiadające ograniczone funkcje i napisane w języku C#, przetłumaczono na język Python i dopasowano na potrzeby systemu. W prezentowanej ostatecznej wersji, algorytmy zostały pogrupowane na pliki definiujące szachowe struktury danych, algorytmy odczytu i zapisu notacji szachowych, algorytmy walidacji zgodności ruchu szachowego z zasadami gry oraz algorytmy sprawdzające wystąpienie zakończenia gry.

II.4. Opis bazy danych

Baza danych systemu została zaprojektowana w modelu relacyjnym z dodatkowymi elementami opartymi na polach typu JSONB, co umożliwia elastyczne przechowywanie danych konfiguracyjnych oraz dynamiczny rozwój systemu. Strukturę bazy danych zaprezentowano na diagramie związków encji (Rys. 4). Przed przejściem do szczegółowego opisu, warto wspomnieć o wyróżniającej się tabeli ‘Auth_user’. Jest ona częścią standardowego systemu uwierzytelniania, automatycznie tworzonego w backendzie Django, w ramach rozszerzenia ‘django.contrib.sessions’. ‘Auth_user’ jest jedną z sześciu tabel systemu uwierzytelniania i jako jedyna została zaprezentowana

na diagramie, gdyż tylko ona jest połączona poprzez relacje z autorskimi tabelami zdefiniowanymi na potrzeby systemu.



Rys. 4. Diagram związków encji bazy danych systemu

Źródło: Opracowanie własne

Opis bazy danych najlepiej zacząć od tabeli ‘Users’, pełniącej kluczową rolę w systemie ze względu na przechowywanie informacji o użytkownikach systemu. W procesie tworzenia konta, najpierw tworzony jest nowy rekord tabeli ‘Auth_user’, posiadający unikalną nazwę gracza na platformie oraz zaszyfrowane hasło. Natychmiast po utworzeniu tej tabeli tworzony jest rekord oryginalnej tabeli ‘Users’, posiadającej dane użytkownika niezwiązane z autoryzacją, ale połączonej relacją z ‘Auth_user’ poprzez unikalne pole ‘username’. Przy tworzeniu konta, pola ‘created_at’ oraz ‘elo’ tworzone są przez system automatycznie, odpowiednio na podstawie aktualnej daty oraz wartości domyślnej. Pozostałe pola domyślnie są bez wartości i nie są wymagane, gdyż służą jedynie personalizacji i mogą być oglądane przez innych użytkowników w trakcie rozgrywania gier i turniejów szachowych.

Jak widać na podstawie relacji, każdy użytkownik platformy ma możliwość przypisania do swojego konta wielu silników szachowych, przechowywanych w tabeli ‘ChessEngines’. Oczywiście jak wspomniano przy omawianiu przypadków użycia, w bazie danych nie są przechowywane dosłownie algorytmy i kod źródłowy silników, tylko przede wszystkim informacja o adresie URL serwera z silnikiem szachowym udostępnianym przez użytkownika oraz dane personalizacyjne, które podobnie jak dla konta użytkownika, widoczne są dla wszystkich graczy korzystających z platformy.

Tabela ‘Games’ przechowuje dane o grach szachowych, natomiast tabela ‘Tournaments’ odpowiada za informacje o turniejach. Obie tabele są do siebie podobne i można wśród nich wyróżnić następujące wspólne cechy:

- Pola ‘start_time’ oraz ‘end_time’, na podstawie których system może określić aktualny stan gier i turniejów.
- Pola ‘format_data’ oraz ‘restrictions_data’ w formacie JSONB, przechowujące złożone dane informujące o ustawieniach rozgrywki takich jak czas i inkrement zegarów szachowych oraz typ zawodników (którzy mogą być ludzcy lub komputerowi), a także nakładające wymagania, które muszą być spełniane przez dołączających graczy. Dla tych pól zdecydowano się na przechowywanie danych w formacie JSON, gdyż takie podejście oferuje możliwość rozszerzania funkcji systemu ustawień i ograniczeń rozgrywek, bez konieczności ingerowania w strukturę tabel.
- Pola ‘is_ranked’, ‘chat’ oraz ‘description’ odpowiednio przechowują informacje o tym czy rozgrywka jest rankingowa (czy na podstawie jej wyniku modyfikowany jest ranking ELO), chat rozgrywki, składający się na rozmowy graczy i komunikaty systemowe oraz opis rozgrywki, dostępny do wglądu np. w wyszukiwarce gier i turniejów oczekujących na rozpoczęcie.

Polami charakterystycznymi dla tabeli ‘Games’ są ‘result’ oraz klucze obce przechowujące dane dołączonych użytkowników. Pole ‘result’ to łańcuch znaków posiadający sformatowane dane zawierające wynik partii (1-0, 0-1, lub 1/2-1/2, gdzie pierwsza z liczb zawsze oznacza wynik użytkownika grającego białymi figurami), rodzaj zakończenia partii (np. ‘Stalemate’) oraz punkty elo przyznawane graczom po partii. Wśród kluczy obcych określających zawodników partii można zauważyć aż cztery pola, wynika to z faktu, że dwa z nich są zawsze przeznaczone dla graczy ludzkich, a pozostałe dwa z przyrostkiem ‘subsitute’ są opcjonalnie uzupełniane, gdy gracze zdecydują się na grę silnikiem szachowym.

Tabela ‘Tournament’ posiada charakterystyczne pola ‘created_by_id’, ‘player_count’, ‘name’ oraz ‘winner_name’. Służą one odpowiednio do przechowywania informacji o właścicielu turnieju, sterującego jego rozpoczęciem i listą graczy, maksymalnej liczbie graczy turnieju, wymaganej nazwie turnieju oraz nazwie zwycięzcy turnieju. Należy zauważyć, że w celu poprawnego rejestrowania uczestników oraz gier turniejów zaimplementowano tabele pośrednie

‘TournamentParticipants’ oraz ‘TournamentGames’. Pierwsza z nich przechowuje informacje o uczestnikach w ten sam sposób co tabela ‘Games’, czyli posiada dwa klucze obce - jeden dla użytkownika i drugi opcjonalny dla jego silnika szachowego. Rekordy uczestników turnieju posiadają też pole ‘score’ reprezentujące liczbę punktów zdobytych przez uczestnika w ramach turnieju. Druga tabela pośrednia posiada jedynie niezbędne pola do zapewnienia relacji wiele-do-wielu między tabelą ‘Tournaments’, a ‘Games’.

Ostatnią do omówienia tabelą jest ‘Moves’ przechowująca dane o pozycjach i ruchach, które miały miejsce w ramach partii szachowej. W tej tabeli zdecydowano się zaimplementować klucz główny złożony na podstawie pól ‘game_id’ oraz ‘move_number’. Oprócz przechowywania numerów ruchów, tabela ‘Moves’ przechowuje informacje o czasie wykonania każdego z nich, ich zapisie w notacji koordynatowej (ang. *Coordinate Notation*) oraz SAN (ang. *Short Algebraic Notation*), a także pozycji na planszy powstałej w ramach ruchu w notacji FEN (ang. *Forsyth–Edwards Notation*). Specyfika wspomnianych notacji zostanie dokładnie opisana później w ramach omawiania algorytmów szachowych, gdyż są one głównym źródłem danych, przetwarzanych w ich ramach.

II.5. Mechanizmy bezpieczeństwa

W ramach zabezpieczenia projektu wdrożono różnorodne mechanizmy bezpieczeństwa we wszystkich warstwach sytemu, przede wszystkim starając się zaadresować zagrożenia prezentowane w pracy autorów [6].

W warstwie prezentacji systemu wykonanej w Angular zdano się jedynie na wbudowaną ochronę przed atakami XSS (ang. *Cross-Site Scripting*), polegającymi na osadzeniu w treści strony złośliwego kodu. Angular automatycznie przeprowadza sanityzację danych wejściowych i wyjściowych, co ogranicza możliwość wstrzykiwania kodu JavaScript.

Backend wykonany w Django posiada implementację kluczowych mechanizmów bezpieczeństwa. Składają się one na:

- Tokeny JWT (ang. *JSON Web Token*) – do uwierzytelniania użytkowników zastosowano tokeny dostępu i odświeżania, które przekazywane są użytkownikom przy logowaniu się na stronę. W trakcie korzystania z witryny tokeny dostępu przesyłane są w nagłówkach żądań, a następnie po stronie backendu podlegają weryfikacji. Tokeny odświeżania wykorzystywane są do pozyskiwania nowych tokenów dostępu, gdy skończy się ich okres ważności.
- CORS (ang. *Cross-Origin Resource Sharing*) – dodano mechanizm Django polegający na udostępnianiu swojego API jedynie zaufanym domenom, zdefiniowanym w konfiguracji serwera. Dzięki temu tylko autoryzowane frontendowe aplikacje mogą komunikować się z backendem.

- Szyfrowanie haseł – zaimplementowano szyfrowanie haseł użytkowników za pomocą algorytmu ‘PBKDF2’, który jest odporny na ataki słownikowe (ang. *Dictionary attack*) i siłowe (ang. *Brute-force attack*). Przechowywanie haseł w zaszyfrowanej formie gwarantuje, że nawet w przypadku nieautoryzowanego dostępu do bazy danych hasła użytkowników pozostają bezpieczne.
- Ograniczanie szybkości (ang. *Rate limiting*) – dodano mechanizm ograniczający liczbę żądań jaką można wykonać w określonym czasie dla punktów końcowych związanych z autoryzacją użytkownika. To zabezpieczenie stanowi dodatkową ochronę przed atakami siłowymi.
- Walidacja haseł – zastosowano domyślne walidatory Django, wymuszające minimalną długość i złożoność haseł tworzonych przez użytkowników.
- Bezpieczne zapytania - system korzysta z Django ORM (ang. *Object-Relational Mapping*), co zapewnia ochronę przed atakami SQL Injection. ORM tłumaczy zapytania napisane w Pythonie na bezpieczne zapytania SQL, automatycznie sanitizując dane wejściowe i unikając wstrzykiwania nieautoryzowanych instrukcji SQL. Wszystkie operacje na bazie danych są realizowane przez zdefiniowane modele, co dodatkowo ogranicza możliwość wykonania błędnych lub niebezpiecznych zapytań.

Baza danych nie posiada szczególnych zabezpieczeń, jednak dostęp do niej został ograniczony jedynie do uprawnionych użytkowników. W konfiguracji systemu użyto zmiennych środowiskowych zdefiniowanych w pliku programu Docker ze środowiska wytwórczego, które precyzują dane logowania, takie jak nazwa użytkownika, hasło i nazwa bazy danych. Dodatkowo, w ustawieniach backend’u określono parametry połączenia z bazą danych, co uniemożliwia dostęp nieautoryzowanym podmiotom.

II.6. Charakterystyka środowiska wytwórczego i uruchomieniowego

II.6.1. Charakterystyka środowiska wytwórczego

Środowisko wytwórcze systemu zostało zorganizowane z wykorzystaniem nowoczesnych narzędzi i technologii, które wspierają proces tworzenia, testowania oraz debugowania aplikacji.

Visual Studio Code od firmy Microsoft jest głównym edytorem, z którego korzystano w procesie programowania systemu. Dzięki rozbudowanemu ekosystemowi rozszerzeń oraz funkcjom ułatwiającym pracę, VSCode umożliwia szybkie i efektywne tworzenie kodu zarówno frontendowego, jak i backendowego. Do pracy z Angular zastosowano rozszerzenie ‘Angular Language Service’, które zapewnia podpowiedzi składni, autouzupełnianie kodu oraz wykrywanie błędów w czasie rzeczywistym. Rozszerzenie Python wspiera rozwój backendu opartego na Django, oferując narzędzia takie jak automatyczne formatowanie, autouzupełnianie oraz debugging. Warto również wspomnieć o zintegrowanym terminalu, który umożliwia uruchamianie serwerów,

kompilację kodu oraz zarządzanie kontenerami Dockera bezpośrednio z poziomu edytora. Chociaż vscode jest prosty w użyciu, to jego dokumentacja [7] okazała się kolejnym dużym atutem.

PgAdmin jest narzędziem wykorzystywanym do zarządzania bazą danych PostgreSQL. Dzięki intuicyjnemu interfejsowi graficznemu umożliwia łatwe przeglądanie struktury bazy danych, wykonywanie zapytań SQL oraz analizę wyników. Narzędzie wspiera również import i eksport danych, co ułatwia testowanie i migrację. Całość tego narzędzia została również skrupulatnie udokumentowana [8].

Google Chrome pełni funkcję głównej przeglądarki używanej do testowania oraz debugowania aplikacji. Dzięki wbudowanym narzędziom deweloperskim możliwa jest szczegółowa analiza kodu HTML, CSS oraz TypeScript, inspekcja zapytań sieciowych oraz monitorowanie wydajności strony.

II.6.2. Charakterystyka środowiska uruchomieniowego

Za środowisko uruchomieniowe systemu odpowiedzialne jest w całości narzędzie Docker. Aplikacja ta służy do konteneryzacji, co oznacza, że pozwala na uruchamianie aplikacji w odizolowanych środowiskach zwanych kontenerami. W przeciwieństwie do maszyn wirtualnych, kontenery działają bezpośrednio na jądrze systemu operacyjnego hosta, co czyni je lżejszymi i bardziej efektywnymi pod względem zużycia zasobów. Dzięki Dockerowi możliwe jest automatyczne pobieranie wszystkich niezbędnych programów i zależności zdefiniowanych w plikach konfiguracyjnych, takich jak ‘Dockerfile’ i ‘docker-compose.yml’. To podejście minimalizuje problemy wynikające z różnic w konfiguracji środowiska na różnych maszynach, zapewniając spójność i powtarzalność procesów wytwarzania oprogramowania, dodatkowo bardzo upraszczając proces instalowania i odinstalowywania projektu. Bardzo ważnym aspektem Dockera jest możliwość łatwego uruchamiania projektu za pomocą prostych poleceń lub interfejsu graficznego, co znacząco przyspiesza pracę nad projektem. Ponadto wbudowane w kontenery narzędzia, takie jak dostęp do konsol i logów, wspierają debugowanie aplikacji oraz umożliwiają szybkie diagnozowanie problemów. Obsługa Dockera nie okazała się trudna dzięki szczegółowej dokumentacji [9].

Rozdział III. Opis implementacji

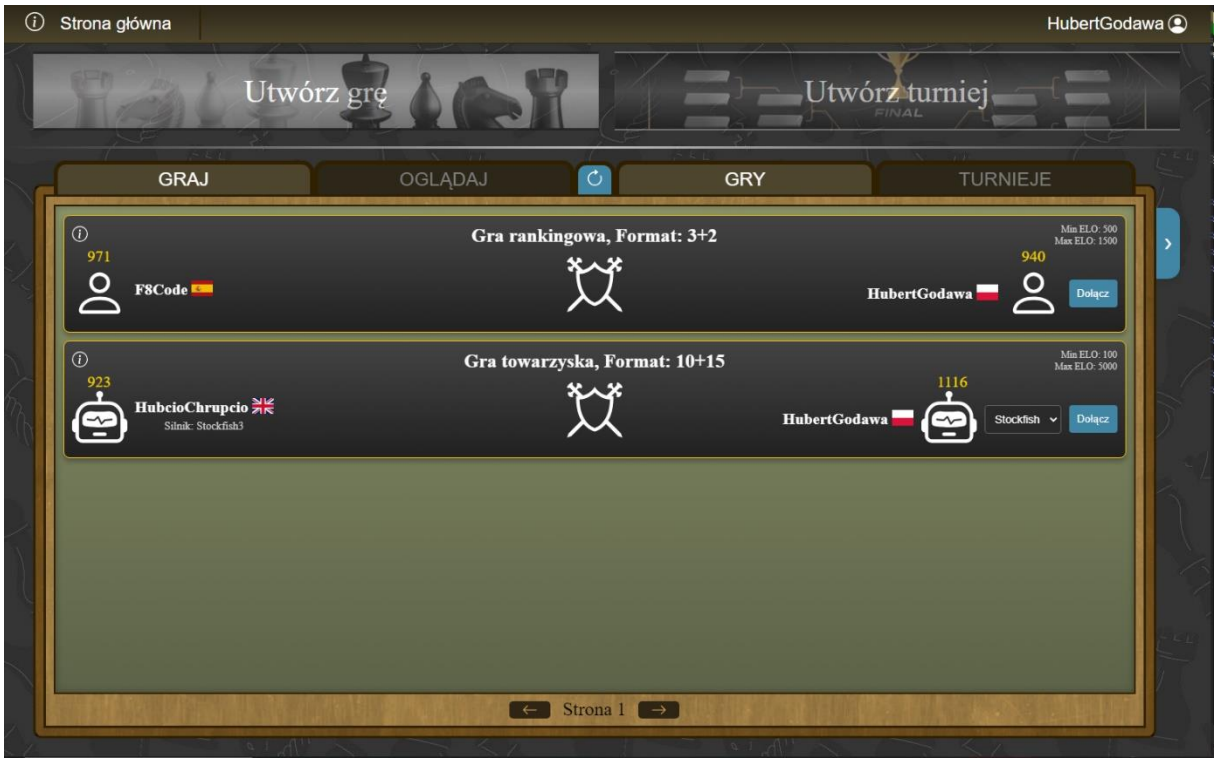
III.1. Interfejs użytkownika

Końcowa aplikacja wytworzona w ramach projektu składa się z siedmiu głównych stron:

- Strona logowania
- Strona główna (z listą gier i turniejów)
- Strona turnieju
- Strona gry szachowej
- Strona profilu użytkownika
- Strona zarządzania silnikami szachowymi
- Strona informacyjna

Ze względu na trywialność strony logowania, jej szczegółowy opis zostanie pominięty, gdyż jej interfejs jest minimalny i pozwala jedynie na wykonywanie prostych operacji rejestrowania i logowania się użytkownika na stronie.

III.1.1. Strona główna aplikacji



Rys. 5. Strona główna aplikacji

Źródło: Opracowanie własne

Na rysunku (Rys. 5) przedstawiono główną stronę aplikacji, która jest pierwszą rzeczą, którą widzi każdy zalogowany i anonimowy użytkownik. Z poziomu tej strony

można uzyskać dostęp do najważniejszych funkcji platformy związanych z tworzeniem i dołączaniem się do gier lub turniejów, a także obserwowaniem rozgrywek w toku.

Na górze strony widać dwa duże guziki oznaczone napisami ‘Utwórz grę’ oraz ‘Utwórz turniej’, otwierające wyskakujące formularze (ang. *pop-up form*) umożliwiające utworzenie rozgrywki wybranego typu. Oba formularze pozwalają na dostosowanie ustawień i opisu rozgrywki. W przypadku zadeklarowania przez twórcę zagrania z użyciem silnika szachowego, musi on wybrać odpowiedni silnik z aktywowanych przez niego wcześniej na stronie.

Pozostały obszar strony zajmuje lista gier i turniejów wyróżniająca się zielonym tłem. Na jej górnej krawędzi widać dwie pary zakładek, które osobno kontrolują wyświetlane obiekty. Na przedstawionym rysunku zaznaczone są zakładki ‘GRAJ’ oraz ‘GRY’, co oznacza, że wyświetlane są gry oczekujące na dołączenie się przeciwnika. W przypadku zmiany zakładki z lewej strony z ‘GRAJ’ na ‘OGLĄDAJ’ wyświetlane są partie szachowe będące w trakcie rozgrywania, do których można się dołączyć jako obserwator. Para zakładek ‘GRAJ’ oraz ‘OGLĄDAJ’ zachowuje się tak samo przy zmianie zakładki z prawej strony z ‘GRY’ na ‘TURNIEJE’, z tą różnicą, że są wtedy wyświetlane karty turniejów. Pośrodku par zakładek widoczny jest niebieski guzik z ikoną krzywej strzałki, służący do odświeżenia listy, zaś z prawej strony zakładek niebieski guzik z prostą strzałką rozwijający panel umożliwiający sortowanie i filtrowanie wyników według zadanych kryteriów. Same karty gier i turniejów wyświetlane na liście posiadają opis (opcjonalnie dodawany przez twórcę), który pojawia się po najechaniu na ikonę informacji w lewym-górnym rogu każdej z kart. Każdy obiekt listy dodatkowo posiada informacje o trybie punktacji i formacie czasowym rozgrywki, a także ograniczeniach ELO jakie musi spełniać osoba dołączająca się. Co najważniejsze, karty posiadają z lewej strony informację twórcy, składające się na rodzaj gracza określany ikoną człowiek/robot, jego ranking ELO, nazwę, nazwę jego silnika (jeśli uczestniczy w grze) oraz opcjonalną narodowość. Z prawej strony karty widać informacje użytkownika, który przegląda listę. Na dole listy umieszczono guziki paginacji do przewijania stron listy, która mieści maksymalnie dziesięć obiektów. Na samej górze strony widoczna jest listwa (ang. *navbar*) z zakładkami strony, która jest stałym elementem interfejsu użytkownika i umożliwia łatwe poruszanie się po stronach platformy.

III.1.2. Strona turnieju



Rys. 6. Strona nierozpoczętego turnieju

Źródło: Opracowanie własne

Na rysunku (Rys. 6) widać ekran strony turnieju w stanie sprzed jego rozpoczęcia. Strona ta posiada trzy główne interfejsy oznaczone tytułami. Po lewej znajduje się chat, na którym wypisywane są informacje systemowe oraz rozmowy między zawodnikami. Z prawej strony jeden nad drugim widać interfejsy pokazujące listę uczestników oraz gry rozegrane w ramach turnieju (ten interfejs rzecz jasna przed rozpoczęciem turnieju nie dostarcza jeszcze żadnych informacji, jest on poniekąd przygotowany przed przejściem turnieju do fazy rozgrywek). W części z informacjami zawodników można zauważyć guziki ‘Usuń’, ‘Usuń miejsce’ oraz ‘Rozpocznij turniej’, świadczące o tym, że rysunek przedstawia ekran turnieju z perspektywy jego właściciela, który posiada dodatkowe funkcje opisane w nazwach guzików. Warto zauważyć, że listwa na górze strony została poszerzona o dodatkową zakładkę ‘Turniej’. Dzieje się tak automatycznie, gdy gracz dołączy do turnieju jako uczestnik. Dzięki tej zakładce zawodnik turnieju może w dowolnym momencie jego trwania przemieszczać się po innych sekcjach strony, w tym oglądać inne rozgrywki, zachowując możliwość szybkiego powrotu do rozgrywki, w której sam uczestniczy. Zakładka ‘Turniej’ na listwie górnej znika po kliknięciu guzika na dole strony ‘Wyjdź z turnieju’ oraz automatycznie po zakończeniu turnieju.



Rys. 7. Strona zakończonego turnieju

Źródło: Opracowanie własne

Na rysunku (Rys. 7) przedstawiono ekran turnieju widoczny po jego zakończeniu. W przeciwieństwie do poprzedniego rysunku, na liście rozegranych gier znajdują się obiekty posiadające informacje o ukończonych rozgrywkach, które można kliknąć w celu dalszej analizy. Warto zaznaczyć, że w trakcie rozgrywania turnieju, na liście widoczne są też partie szachowe, które nie zostały jeszcze ukończone, a kliknięcie ich pozwala obserwować je w czasie rzeczywistym. Również w interfejsie chatu pojawiły się zmiany względem poprzedniego rysunku w postaci nowych wiadomości systemowych informujących o zmianach stanu turnieju. Sama lista zawodników także jest inna, w jej tytule widać teraz nazwę zwycięzcy, a w zawartości punktacje zawodników uzyskane w partiach z innymi graczami oraz wynik końcowy.

Prezentację ekranu turnieju w trakcie rozgrywania pominięto, gdyż posiada on interfejsy identyczne do tych z rysunku (Rys. 7) z wyjątkiem nazwy zwycięzcy na górze listy zawodników.

III.1.3. Strona gry szachowej



Rys. 8. Strona gry szachowej

Źródło: Opracowanie własne

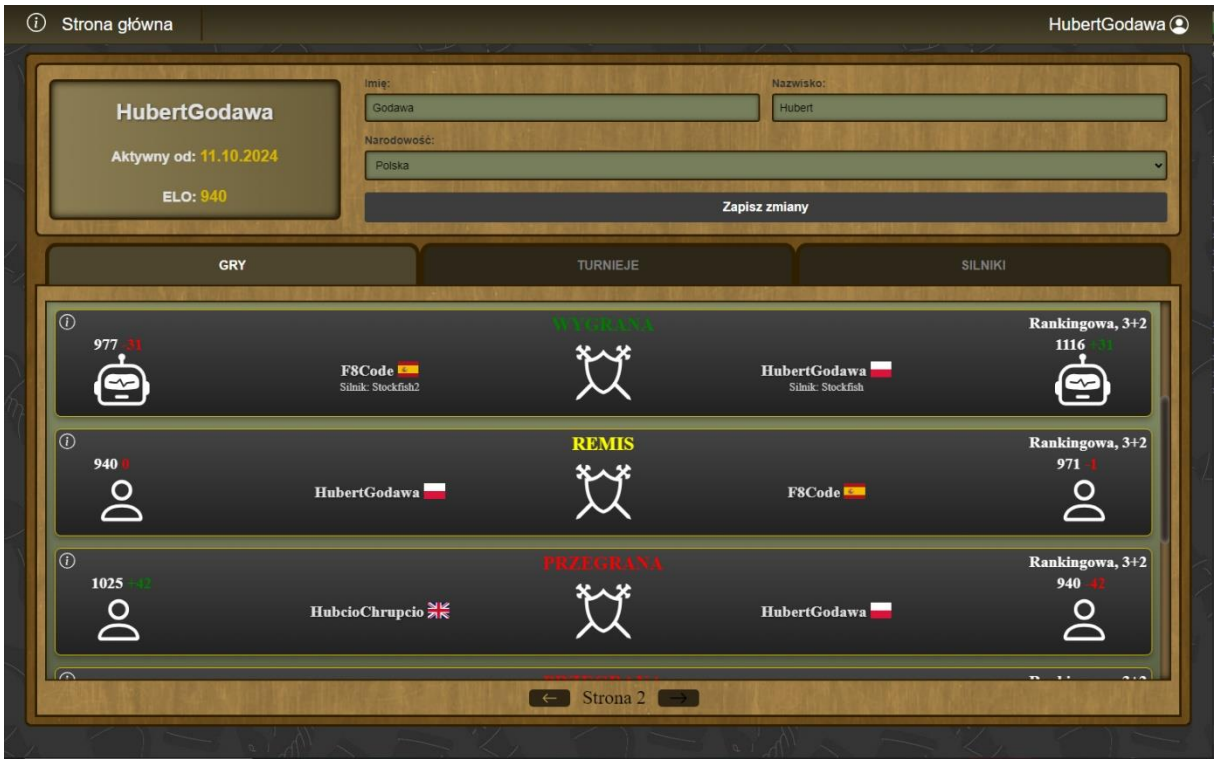
Rysunek (Rys. 8) przedstawia najbardziej rozbudowany ekran aplikacji, czyli ekran gry szachowej. Składa się na niego aż pięć komponentów Angular, czyli pięć obiektów posiadających własną logikę i interfejs graficzny, są to:

- Komponent zewnętrzny – komponent stanowiący kontener dla wszystkich elementów strony, organizujący je w pożądany układ i stanowiący główny węzeł wymiany danych. Definiuje też własne interfejsy, jak na przykład guzik ‘Wyjdź z gry’ na dole strony, czy też informacje zawodników nad i pod planszą szachową oraz stany ich zegarów szachowych.
- Komponent chatu – komponent wtórnie wykorzystany z chatu dostępnego na stronie turnieju, w celu uniknięcia redundancji kodu. Znajduje się z lewej strony ekranu, posiada systemowe komunikaty o zmianie stanu gry oraz znaczących wydarzeniach, umożliwiający dodatkowo rozmowy między graczami.
- Komponent szachownicy – komponent znajdujący się pośrodku strony, stanowiący jej najważniejszy element, gdyż umożliwia rozgrywkę partii. Pozwala na wykonywanie ruchów poprzez przeciąganie figur, gdy gracz ma do tego prawo (na przykład, gdy jest jego tura), co decydowane jest przez komponent zewnętrzny strony. Posiada logikę podświetlającą pole źródła i celu ostatniego wykonanego ruchu na kolor zielony lub czerwony,

w zależności od powodzenia jego walidacji oraz sygnalizuje wszystkie ruchy różnorodnymi dźwiękami, poprawiającymi interaktywność rozgrywki.

- Komponent listy ruchów – komponent znajdujący się z prawej strony ekranu, posiada informacje o liście ruchów szachowych w notacji SAN oraz zapis obecnej na planszy pozycji w notacji FEN. Guziki na górze komponentu pozwalają na przechodzenie pomiędzy pozycjami rozegranymi w ramach gry, guziki brzegowe przesuwają pozycję o jeden ruch w tył/przód, środkowy przycisk powraca do ostatniego ruchu rozgrywki. Warto zwrócić uwagę na subtelny guzik z prawej strony okna notacji FEN, umożliwiający skopiowanie jej do schowka, w celu wklejenia do zewnętrznych narzędzi analitycznych.
- Komponent guzików akcji – komponent wyświetlany pod komponentem listy ruchów, składający się z trzech guzików oznaczonych ikonami flagi, uścisku dłoni oraz strzałki. Pozwalają one odpowiednio na poddanie się, zaproponowanie/zatwierdzenie remisu oraz zaproponowanie/zatwierdzenie cofnięcia ostatniego ruchu. Wszystkie akcje potwierdzane są podświetleniem kontrolki na kolor czerwony/żółty/zielony, w zależności od guzika. Dla przykładu guzik z ikoną poddania partii podświetla się na czerwono przy pierwszym wciśnięciu, a dopiero za drugim poddaje partię. W przypadku kliknięcia go tylko raz, po dziesięciu sekundach jego podświetlenie znika i ponowna próba poddania się znów będzie wymagać dwukliku.

III.1.4. Strona profilu użytkownika

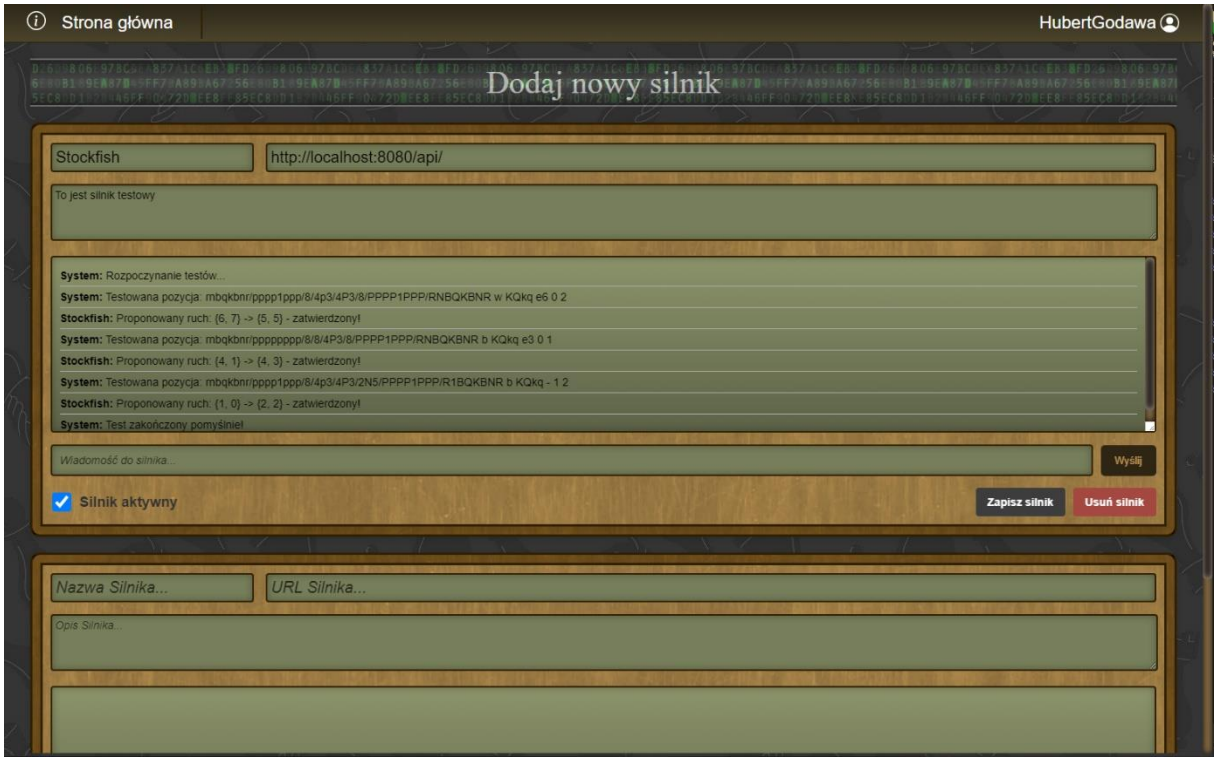


Rys. 9. Strona profilu użytkownika

Źródło: Opracowanie własne

Na rysunku (Rys. 9) widać ekran profilu użytkownika. Składa się on na sekcję informacji o profilu na górze strony oraz sekcję z listą gier i turniejów ukończonych przez użytkownika, a także jego silników szachowych, sterowaną poprzez zakładki umieszczone na jej szczycie. Elementy listy podobnie jak na stronie głównej są interaktywne i klikanie ich przenosi do ekranu wybranej rozgrywki, a w przypadku kart silników szachowych do ekranu zarządzania silnikami szachowymi.

III.1.5. Strona zarządzania silnikami szachowymi



Rys. 10. Strona zarządzania silnikami szachowymi

Źródło: Opracowanie własne

Na stronę zarządzania silnikami szachowymi przedstawioną na rysunku (Rys. 10) można się dostać poprzez listę silników szachowych na stronie profilu użytkownika, bądź w szybszy sposób - poprzez najechanie na nazwę konta na listwie górnej (z prawej strony), co skutkuje rozwinięciem dodatkowych opcji, w tym zakładki przenoszącej na omawianą stronę. W tej części platformy guzikiem ‘Dodaj nowy silnik’ znajdującym się na górze ekranu można utworzyć nowy obiekt silnika szachowego, który da się zapisać po podaniu jego nazwy i adresu URL serwera udostępniającego jego funkcje. Dodatkowo każdy silnik szachowy posiada opcjonalne pole opisu, które użytkownik może wypełnić w celu poprawienia organizacji i przypominania o cechach szczególnych serwerów. Najważniejszym elementem każdej instancji silnika szachowego na tej stronie jest okno chatu z silnikiem, umożliwiającym wysyłanie wiadomości na punkt końcowy ‘/chat’ jego serwera i odczytywanie odpowiedzi. Bardzo ważny jest też przycisk wyboru (ang. *checkbox*) aktywowania silnika w lewym dolnym rogu każdej

instancji, przeprowadzający serię testów jawnie widoczną na chacie, która po udanym zakończeniu aktywuje silnik na stronie i umożliwia wykorzystywanie go w rozgrywkach. Silniki użytkownik może w każdej chwili edytować i usuwać.

III.1.6. Strona informacyjna



Rys. 11. Strona informacyjna

Źródło: Opracowanie własne

Ostatnią stroną projektu jest strona informacyjna (Rys. 11), dostępna po naciśnięciu ikony informacji po lewej stronie górnej listwy platformy. Strona ta zawiera informacje dla deweloperów korzystających ze strony, o wymaganiach jakie powinien spełniać silnik szachowy, w tym linki prowadzące do stron z informacjami pomocniczymi oraz przykładową implementacją serwera z silnikiem szachowym, wykonanym w ramach projektu.

III.2. Szachowe algorytmy walidacji ruchu

Algorytmy walidacji ruchu szachowego są najważniejszym elementem projektu, zapewniającym zgodność rozgrywek z regułami szachowymi. W ramach systemu zaprojektowano i zaimplementowano własne rozwiązania, które początkowo utworzono w języku C# jako część projektu szachowego w grze komputerowej ‘Space Engineers’. W trakcie wytwarzania platformy, algorytmy te zostały przetłumaczone na język Python i dostosowane do specyfiki backendu Django oraz reszty systemu. Decyzja o zaprojektowaniu własnych algorytmów, zamiast korzystania z dostępnych bibliotek lub gotowych rozwiązań, przyniosła szereg korzyści. Przede wszystkim zapewniła pełną

transparentność i kontrolę nad implementacją, co ma szczególne znaczenie w przypadku projektów takich jak ten, gdzie różne aspekty systemu, od kontroli poprawności ruchów, po analizę zakończenia gry, muszą działać spójnie i przewidywalnie. Dodatkowo własne algorytmy ułatwiają utrzymanie kodu oraz wprowadzanie zmian, takich jak dodanie wariantów szachowych, które mimo różnic w zasadach bazują na fundamentach klasycznych reguł gry.

Znaczenie algorytmów walidacji ruchu jest szczególnie ważne w kontekście zapewnienia uczciwej rozgrywki między ludźmi oraz w interakcjach z silnikami szachowymi. Weryfikacja poprawności ruchów chroni system przed przypadkowymi błędami oraz próbami oszustwa, a w przypadku silników szachowych, które mogą działać niestabilnie (szczególnie gdy bazują na nieliniowych modelach sieci neuronowych), algorytmy te zapewniają dodatkową warstwę ochrony i kontroli.

Przestawione zostaną trzy główne aspekty związane z walidacją ruchów szachowych: struktury danych wykorzystywane w procesie walidacji, odczyt notacji zawierających informacje o aktualnej pozycji na planszy oraz zagranego ruchu oraz same algorytmy walidacji podzielone na sekcje względem etapów.

III.2.1. Struktury danych

Podstawą działania algorytmów walidacji ruchów szachowych są odpowiednio zaprojektowane struktury danych, które przechowują kluczowe informacje o stanie gry oraz szczegóły dotyczące wykonywanych ruchów. Struktury te zostały opracowane w sposób umożliwiający zarówno łatwą interpretację danych wejściowych, jak i ich przetwarzanie w algorytmach walidacji.

Kod. 1. Enumeracja ‘ChessPiece’

```
1   class ChessPiece(Enum):
2       Pawn = 1
3       Rook = 2
4       Knight = 3
5       Bishop = 4
6       Queen = 5
7       King = 6
```

Źródło: Opracowanie własne

Omawianie struktur warto zacząć od klasy ’ChessPiece’ (Kod. 1), która jest enumeracją reprezentującą wszystkie rodzaje figur szachowych: piony, wieże, skoczki, gońce, hetmani i króle. Struktura ta została zainspirowana reprezentacją bierek na planszy przez autorów [10], a jej rozbudowanie do Enum poprawia intuicyjność i znacząco ułatwia zarówno programowanie, jak i debugowanie algorytmów.

Kod. 2. Klasa ‘ChessFigure’

```
1   class ChessFigure:
2       def __init__(self, type: ChessPiece, is_white: bool):
3           self.type = type
4           self.is_white = is_white
5
6       def __copy__(self):
7           return ChessFigure(self.type, self.is_white)
```

Źródło: Opracowanie własne

Klasa ‘ChessFigure’ (Kod. 2) stanowi naturalne rozwinięcie klasy ‘ChessPiece’, dodając do niej informację o białym lub czarnym kolorze jaki może posiadać figura szachowa. Klasa ta jest fundamentalnym elementem reprezentacji figur na planszy w systemie. Jej istotną cechą jest możliwość łatwego kopiowania obiektów za pomocą metody ‘__copy__’, pozwalającej na łatwe symulowanie różnych ruchów na planszy.

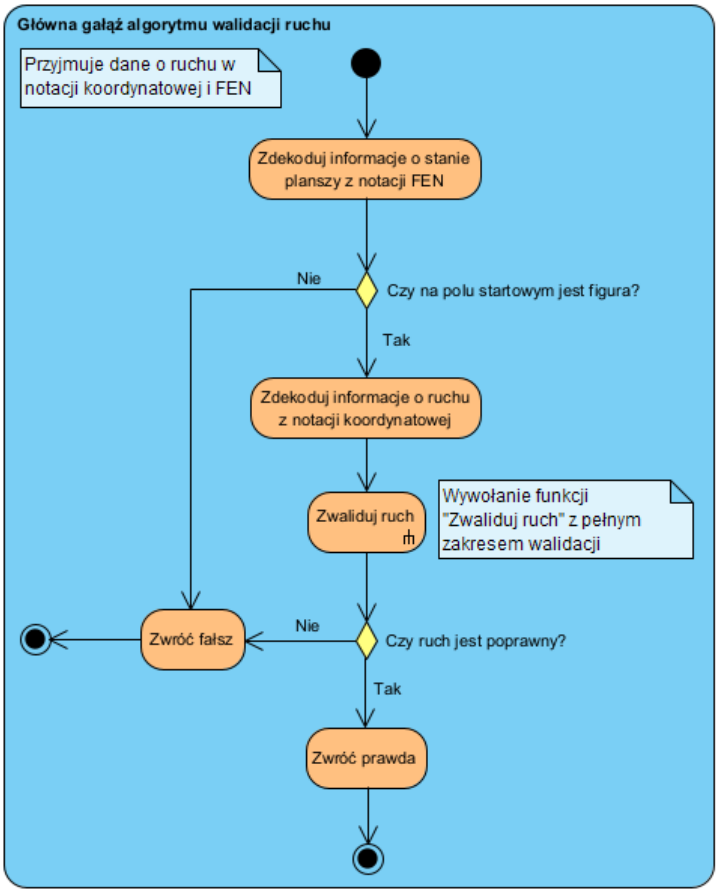
Kod. 3. Struktura ‘MoveInfo’

```
1   class MoveInfo:
2       def __init__(self):
3           self.source_x: int = 0
4           self.source_y: int = 0
5           self.target_x: int = 0
6           self.target_y: int = 0
7           self.promoted_to: Optional[ChessPiece] = None
8
9           self.chess_board:  List[List[Optional[ChessFigure]]]  =
    [[None for _ in range(8)] for _ in range(8)]
10          self.white_turn: bool = True
11          self.can_white_s_castle: bool = False
12          self.can_white_l_castle: bool = False
13          self.can_black_s_castle: bool = False
14          self.can_black_l_castle: bool = False
15          self.en_passant_target:   Optional[Tuple[Optional[int],
    Optional[int]]] = (None, None)
16          self.half_move_clock: int = 0
17          self.move_number: int = 1
18
19          self.source_piece: Optional[ChessFigure] = None
20          self.target_piece: Optional[ChessFigure] = None
21          self.distance_x: int = 0
22          self.distance_y: int = 0
23          self.direction_x: int = 0
24          self.direction_y: int = 0
25          self.is_enpassant: bool = False
```

Źródło: Opracowanie własne

Klasa ‘MoveInfo’ (Kod. 3) stanowi centralny element systemu odpowiedzialny za gromadzenie i udostępnianie danych niezbędnych do analizy i walidacji ruchów szachowych, powstała ona jako rozbudowanie obiektowej struktury danych wykorzystywanej przez autora projektu [11]. To właśnie dzięki niej możliwe jest przetwarzanie wszystkich istotnych informacji o stanie gry, takich jak współrzędne pól początkowego i docelowego, aktualny układ planszy czy możliwość wykonania specjalnych ruchów, takich jak roszada lub bicie w przelocie. Struktura ta pełni rolę głównego źródła danych nie tylko dla algorytmów walidacyjnych, ale także dla algorytmów badania zakończenia gry i generowania notacji szachowych. ‘MoveInfo’ oprócz danych zawartych bezpośrednio w notacjach przekazywanych programowi, posiada obliczone informacje statystyczne ruchu takie jak jego długość i kierunek, co umożliwia wygodne i intuicyjne korzystanie z danych podczas iteracyjnych operacji na planszy, takich jak skanowanie pól w celu analizy dostępnych ruchów.

III.2.2. Odczyt danych

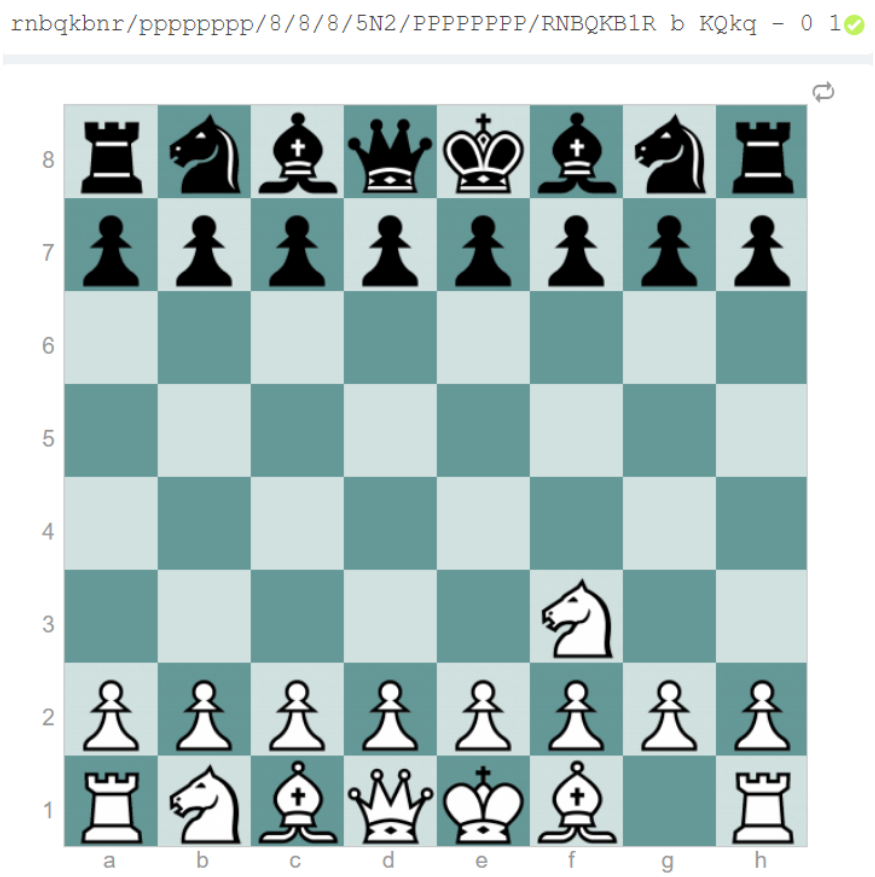


Rys. 12. Diagram aktywności głównej gałęzi algorytmu walidacji ruchu szachowego

Źródło: Opracowanie własne

Na diagramie aktywności (Rys. 12) przedstawiono główną gałąź algorytmu walidacji ruchu szachowego, będącą punktem wejścia i wyjścia dla poszczególnych fragmentów kodu. Na rysunku widać, że przed przejściem do samej walidacji ruchu

odbywa się odczyt danych i inicjalizacja wcześniej przedstawionych struktur szachowych na ich podstawie. W skład odczytu danych wchodzą aktywności „Zdekoduj informacje o stanie planszy z notacji FEN" oraz „Zdekoduj informacje o ruchu z notacji koordynatowej". Na połączenie tych dwóch notacji zdecydowano się na podstawie sposobu badania zmian na planszy szachowej przedstawionej przez autora [12]. Warto zauważyć, że przed dokładnym zdekodowaniem notacji koordynatowej sprawdzane jest czy plansza odczytana z notacji FEN w ogóle posiada figurę na polu źródłowym zawartym w notacji koordynatowej. Jeśli takiej figury nie ma, to algorytm natychmiast przerywa pracę w celu zmniejszenia ilości obliczeń i zwraca niepowodzenie walidacji.



Rys. 13. Przykład notacji FEN

Źródło: https://www.redhotpawn.com/chess/chess-fen-viewer.php

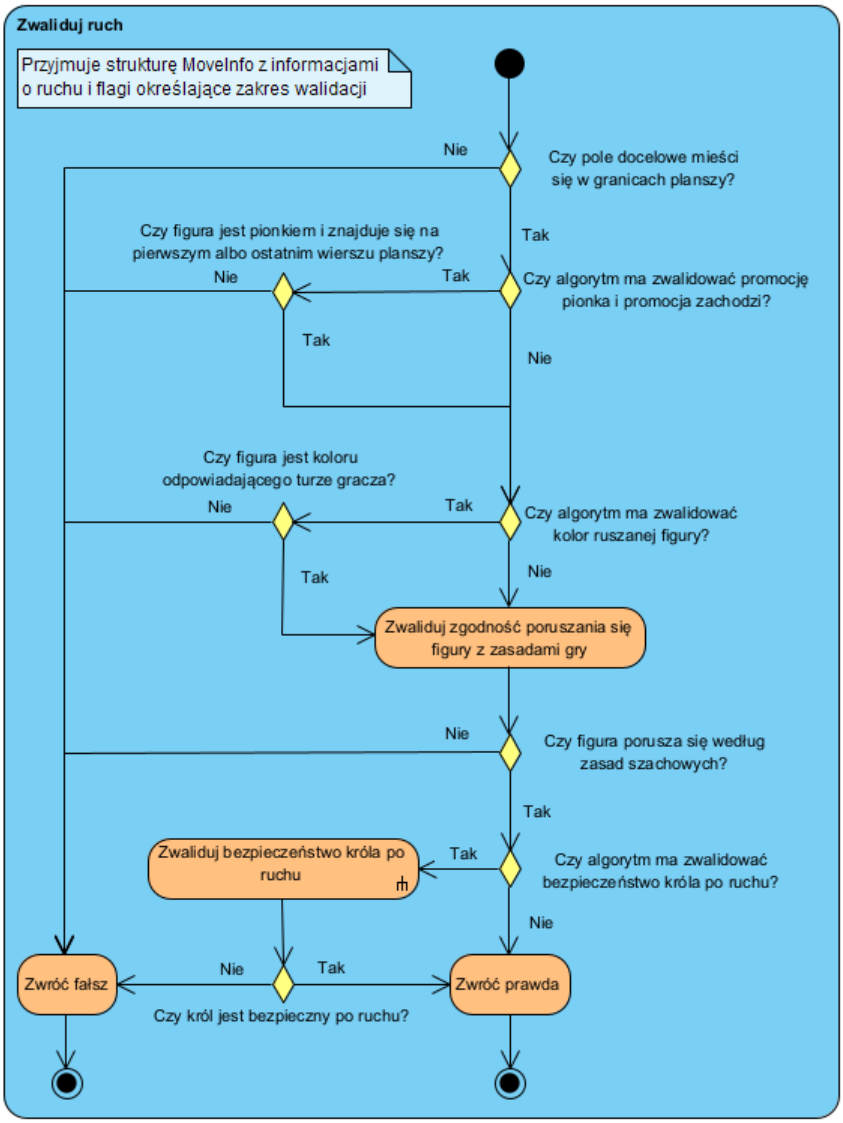
Rysunek (Rys. 13) przedstawia przykładową notację FEN wraz z reprezentacją opisywanej przez nią planszy. W zapisie przedstawionym na górze ilustracji każdy rząd planszy jest reprezentowany jako ciąg znaków tekstowych, gdzie każdy znak odpowiada figurze lub pustemu polu na planszy. Figury białe oznaczane są dużymi literami (P, R, N, B, Q, K), natomiast figury czarne – literami małymi (p, r, n, b, q, k). Liczby w notacji symbolizują ciąg pustych pól, np. 8 oznacza osiem pustych pól w rzędzie. Rzędy planszy oddzielone są ukośnikami (/), zaczynając od rzędu znajdującego się najbliżej gracza

czarnych i kończąc na rzędzie gracza białych. Dodatkowe sekcje po opisie układu planszy zawierają:

- Tura gracza – pojedynczy znak (w dla białych lub b dla czarnych), wskazujący, który gracz ma wykonać ruch.
- Roszady – litery K, Q, k, q oznaczają dostępność roszad odpowiednio po stronie: białej królewskiej, białej hetmańskiej, czarnej królewskiej, czarnej hetmańskiej. Jeśli żadna roszada nie jest możliwa, w sekcji tej występuje ‘-‘.
- Bicie w przelocie – współrzędne pola, które może być celem bicia w przelocie, lub znak ‘-‘, jeśli taki ruch nie jest możliwy.
- Licznik półruchów – liczba półruchów od ostatniego ruchu pionkiem lub bicia, używana do sprawdzenia reguły 50 ruchów.
- Numer ruchu – numer pełnego ruchu, przy czym pierwszy ruch białych oznaczony jest jako 1.

Dodatkowym źródłem danych dla klasy ‘MoveInfo’ (Kod. 3) jest notacja koordynatowa, często wykorzystywana w szachowych systemach informatycznych, będąca uproszczonym formatem zapisu ruchów wykonywanych przez graczy. Składa się ona z pary współrzędnych pól początkowego i docelowego oraz ewentualnej informacji o promocji wykonywanej przez pionek po dotarciu do ostatniego rzędu planszy. Przykładowa notacja koordynatowa może wyglądać następująco: ‘0607q’. Dla tego przykładu gracz wykonał ruch figurą z pola (0, 6) na pole (0, 7), a także doszło do promocji figury na hetmana (q).

III.2.3. Funkcja walidacji ruchu szachowego



Rys. 14. Diagram aktywności funkcji walidacji ruchu szachowego

Źródło: Opracowanie własne

Diagram aktywności (Rys. 14) przedstawia dokładny przebieg funkcji walidacji ruchu. Warto zwrócić uwagę, że funkcja ta służy nie tylko do walidacji poprawności oryginalnego ruchu, ale jest też wykorzystywania analitycznie później do badania różnych cech planszy i dlatego można ją wywoływać z różnym zakresem zadań, który wyjątkowo dla przypadku walidacji ruchu zaproponowanego przez użytkownika jest pełny. Przy pełnym zakresie walidacji sprawdzana jest obecność docelowego pola w granicach planszy, poprawność promocji pionka (jeśli zachodzi), zgodność koloru ruszanej figury z turą gracza, zgodność sposobu poruszania się figury z zasadami gry oraz bezpieczeństwo króla po wykonaniu ruchu (w szachach nie można wykonać ruchu narażającego własnego króla na zbicie). Większość określonych testów polega na sprawdzeniu lub porównaniu ze sobą prostych informacji zawartych w strukturze

‘MoveInfo’, z wyjątkiem aktywności „Zwaliduj zdodność poruszania się figury z zasadami gry” oraz „Zwaliduj bezpieczeństwo króla po ruchu”, które posiadają bardziej skomplikowaną logikę. Aby ją w pełni zrozumieć, wpierw przedstawiona zostanie najważniejsza funkcja pomocnicza, sprawdzająca możliwość poruszenia się wybranych figur na wskazane pole.

III.2.4. Funkcja zliczająca figury mogące wykonać ruch na zadane pole

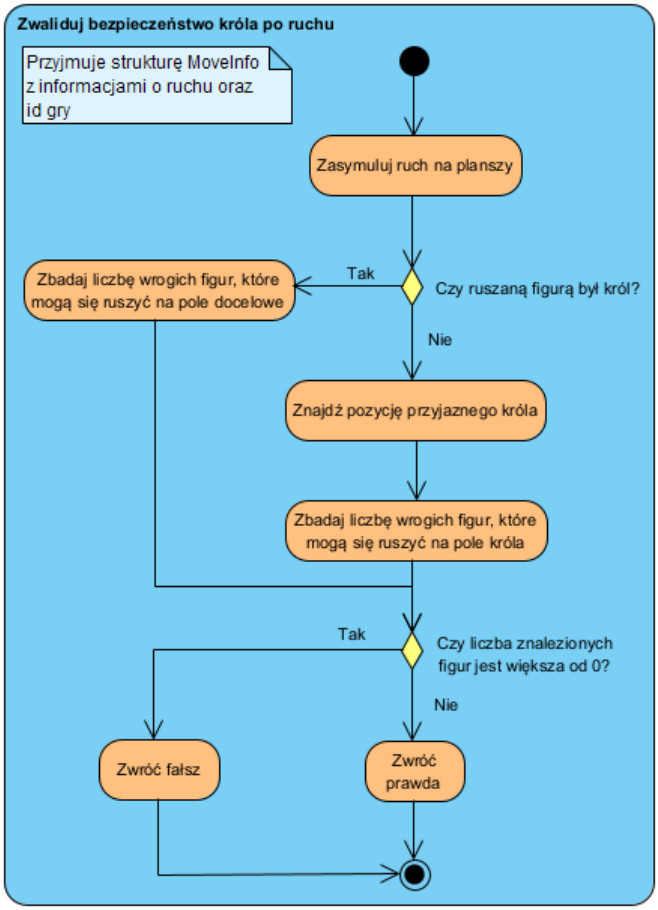
Kod. 4. Funkcja zliczająca figury mogące się ruszyć na zadane pole

```
1   def get_pieces_that_can_go_to_square(target_x, target_y,
    move_info, for_white, validate_king_safety):
2       pieces_that_can_move = []
3
4       simulated_move_info = copy.deepcopy(move_info)
5       for x in range(8):
6           for y in range(8):
7               piece = move_info.chess_board[x][y]
8               if piece is not None and
    piece.is_white==for_white:
9                   simulated_move_info.source_x = x
10                  simulated_move_info.source_y = y
11                  simulated_move_info.target_x = target_x
12                  simulated_move_info.target_y = target_y
13                  if
    is_move_valid_function(simulated_move_info,
    validate_king_safety, False, False):
14                      pieces_that_can_move.append((x, y))
15
16      return pieces_that_can_move
```

Źródło: Opracowanie własne

Zaprezentowany kod (Kod. 4) iteracyjnie przeszukuje wszystkie pola planszy i zwraca wszystkie znalezione figury wybranego koloru mogące przemieścić się na zadane pole. Działanie tego algorytmu opiera się na wywołaniu funkcji walidującej ruch szachowy jedynie z flagą walidacji bezpieczeństwa króla po ruchu, choć ta także jest opcjonalna. Ponieważ funkcja walidująca ruch zawiera kilka testów obowiązkowych, to efektywnie wyszukiwane są wszystkie figury na planszy, które mogą się poruszyć na wybrane pole zgodnie ze swoim sposobem poruszania się, z opcjonalnym uwzględnieniem bezpieczeństwa królu po ich ruchu. Dla bezpieczeństwa operacji na danych zawsze sprawdzane jest czy pole docelowe mieści się w granicach planszy. W następnych algorytmach, przedstawiona funkcja będzie wiele razy wykorzystywana do rozstrzygania ‘zagrożenia pól’, czyli sprawdzania czy na dane pole może się ruszyć dowolna figura przeciwnika.

III.2.5. Funkcja sprawdzająca bezpieczeństwo króla po ruchu



Rys. 15. Diagram aktywności funkcji walidacji bezpieczeństwa króla po ruchu

Źródło: Opracowanie własne

Chociaż ze względu na kolejność wykonywania operacji w algorytmie walidacji ruchu może się to wydawać nieintuicyjne, to przed omówieniem funkcji badającej poprawność poruszania się figur z zasadami, lepiej się skupić na funkcji sprawdzającej bezpieczeństwo króla po ruchu. Przebieg tego algorytmu przedstawiono na diagramie aktywności (Rys. 15). Działanie tej funkcji opiera się w dużej mierze na kodzie (Kod. 4) omówionym wcześniej - jeżeli gracz poruszył się królem, to sprawdzana jest liczba bierek koloru przeciwnika, mogących przejść na pole docelowe tego ruchu, jeśli zaś gracz ruszył się bierką innego typu, to pozycja króla wyszukiwana jest dodatkowym fragmentem kodu i również zliczane są wrogie figury, które mogą na nią przejść. Warto zauważyć, że dla tego kodu, funkcja zliczająca wrogie figury nie sprawdza bezpieczeństwa ich króla po ruchu. Takie postępowanie wynika z faktu, że w szachach król uznawany jest za zagrożonego (szach) niezależnie od tego czy figura, wroga która może się ruszyć na jego pole odsłoni własnego króla czy nie.

III.2.6. Funkcja sprawdzająca poprawność sposobu poruszania się figury

Kod. 5. Funkcje walidacji poprawności poruszania się wybranych figur

```
1   def does_follow_rook_movement_rules(move_info):
2       # Standard rook movement (along straight lines)
3       if move_info.distance_x==0 or move_info.distance_y==0:
4           return is_path_clear(move_info)
5
6       return False
7
8   def does_follow_bishop_movement_rules(move_info):
9       # Standard bishop movement (along diagonals)
10      if abs(move_info.distance_x)==abs(move_info.distance_y):
11          return is_path_clear(move_info)
12
13      return False
14
15  def does_follow_queen_movement_rules(move_info):
16      # Queen moves like a rook or bishop
17      if does_follow_rook_movement_rules(move_info) or
    does_follow_bishop_movement_rules(move_info):
18          return True
19
20      return False
```

Źródło: Opracowanie własne

Ostatnią ważną funkcją wchodzącą w skład procesu walidacji ruchu szachowego jest funkcja sprawdzająca poprawność poruszania się figury względem zasad szachowych. W zależności od typu bierki proces walidacji delegowany jest do odpowiednich funkcji szczegółowych. Przedstawiony fragment kodu (Kod. 5) odpowiedzialny jest za sprawdzanie poprawności ruchu dla wieży, gońca i hetmana. Proces ten polega na sprawdzeniu kierunku poruszania się tych figur, a także w funkcji ‘is_path_clear’, możliwości kolizji z figurami po swojej drodze (wszystkie figury oprócz skoczka mogą przechodzić na docelową pozycję jedynie przez puste pola), a także z przyjazną figurą na polu docelowym (co świadczyłoby o zbiciu przyjaznej bierki, co w szachach jest zabronione). Dla wieży sprawdzane jest czy porusza się po prostej linii, a dla gońca czy ruch jest po skosie. Walidacja hetmana, w celu uniknięcia redundancji kodu, polega na sprawdzeniu czy porusza się on jak wieża lub goniec. Funkcja sprawdzająca ruch skoczka działa na bardzo podobnej zasadzie jak wyżej wymienione, a dla pionka przewidywane są przypadki brzegowe takie jak bicie w przelocie oraz fakt, że może się on poruszać na różne sposoby (prosto, gdy docelowe pole jest puste, lub na ukos, gdy na docelowym polu jest wroga figura). Walidacja ruchu króla przewiduje dodatkowo możliwość wykonania przez niego roszady. W tym przypadku, wykorzystywana jest funkcja zliczająca figury, które mogą się ruszyć na

zadane pole (Kod. 4) w podobny sposób jak w algorytmie sprawdzającym bezpieczeństwo króla po ruchu. W tym wypadku sprawdzane jest bezpieczeństwo pól, przez które musi przejść król aby wykonać roszadę, gdyż zgodnie z zasadami szachów, w trakcie roszady żadne z nich nie może być zagrożone atakami wrogich bierek.

III.3. Szachowe algorytmy rozstrzygania rezultatu ruchu

Algorytm rozstrzygania rezultatu ruchu dostarcza opcjonalną funkcję, z której można skorzystać po udanej walidacji ruchu. Polega ona na detekcji i klasyfikacji zakończenia gry oraz wygenerowania notacji SAN i FEN na podstawie wykonanego ruchu oraz pozycji na planszy, która jest jego rezultatem. Funkcja ta wykorzystywana jest zawsze w trakcie rozgrywek szachowych, w celu zapewnienia ich spójności, ciągłości i poprawności przebiegu, a pomijana jest na przykład w przypadku aktywacji silnika szachowego na platformie, kiedy liczy się sprawdzenie poprawności proponowanych ruchów, a nie ich rezultat.

III.3.1. Struktury danych

Algorytmy do sprawdzania rezultatu ruchu, podobnie jak algorytmy do jego walidacji, posiadają ściśle określone struktury danych, stanowiące źródło informacji do przetwarzania w kodzie.

Kod. 6. Enum ‘DrawType’

```
1       class DrawType(Enum):
2           Stalemate = 1
3           InsufficientMaterial = 2
4           ThreefoldRepetition = 3
5           FiftyMoveRule = 4
```

Źródło: Opracowanie własne

Klasa ‘DrawType’ (Kod. 6) jako enumeracja skupia w sobie rodzaje remisów jakie mogą zajść w partii, czyli: pat, trzykrotne powtórzenie pozycji, brak wystarczającego materiału na planszy do wygrania gry, czy osiągnięcie limitu 50 ruchów bez bicia ani ruchu pionkiem. Podobnie jak z wcześniej przedstawianą enumeracją ‘ChessPiece’, wykorzystanie Enum do reprezentacji remisów ułatwia programowanie i debugowanie kodu, a także organizuje informacje w sposób umożliwiający łatwą rozbudowę systemu w przypadku wprowadzania wariantów szachowych.

Kod. 7. Struktura ‘NotationInfo’

```
class NotationInfo:
    def __init__(self):
        # Algebraic and coordinate notation writing information
        self.source_x: int = 0
        self.source_y: int = 0
        self.target_x: int = 0
        self.target_y: int = 0
        self.promoted_to: Optional[ChessPiece] = None
        self.is_l_castling: bool = False
        self.is_s_castling: bool = False
        self.piece_type: Optional[ChessPiece] = None

        self.is_capture: bool = False
        self.is_enpassant: bool = False
        self.is_check: bool = False

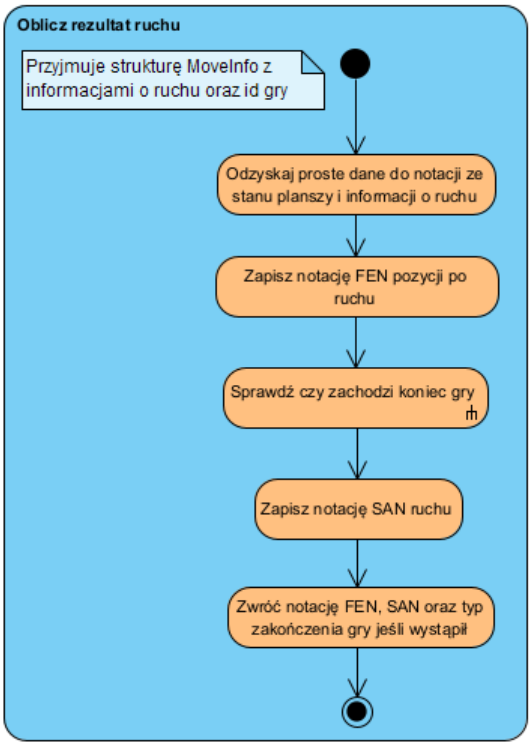
        # FEN notation writing information
        self.chess_board: List[List[Optional[ChessFigure]]] = [[None for _ in range(8)] for _ in range(8)]
        self.white_turn: bool = True
        self.can_white_l_castle: bool = False
        self.can_white_s_castle: bool = False
        self.can_black_l_castle: bool = False
        self.can_black_s_castle: bool = False
        self.en_passant_target: Optional[Tuple[Optional[int], Optional[int]]] = (None, None)
        self.half_move_clock: int = 0
        self.move_number: int = 1

        # Game writing information (Also used for Algebraic notation)
        self.is_checkmate: bool = False
        self.draw_type: Optional[DrawType] = None
```

Źródło: Opracowanie własne

Kluczową strukturą do zapisu notacji szachowych FEN i SAN jest ‘NotationInfo’ (Kod. 7), inicjalizowana w dużej mierze na podstawie wcześniej przedstawionej struktury ‘MoveInfo’. Jak można zauważyć, w ‘NotationInfo’ zachodzi pewna redundancja danych względem ‘MoveInfo’, jednak wyraźne definiowanie i przydzielanie odpowiednich struktur do zadań niesie za sobą korzyść utrzymywania przejrzystości i czytelności kodu, co z kolei przekłada się na jego łatwe utrzymanie i modyfikowanie, a jak wspomniano wcześniej, są to właśnie główne walory przedstawianych algorytmów. Kod struktury podzielono za pomocą komentarzy na sekcje, wskazujące zastosowanie odpowiednich zmiennych.

III.3.2. Główna gałąź algorytmu obliczającego rezultat ruchu

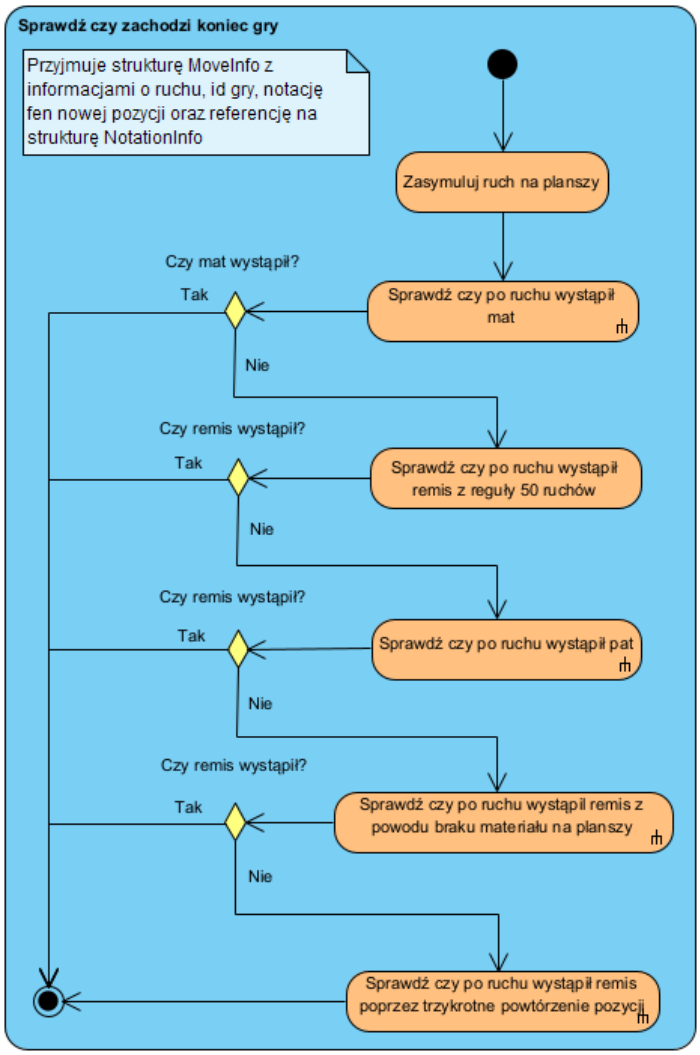


Rys. 16. Diagram aktywności funkcji obliczania rezultatu ruchu

Źródło: Opracowanie własne

Na diagramie aktywności (Rys. 16) przedstawiono główną gałąź algorytmu obliczania rezultatu ruchu, stanowiącą punkt wejścia i wyjścia dla poszczególnych fragmentów kodu i wywołującą kluczowe operacje wykonywane w ramach algorytmu obliczania rezultatu ruchu. Proces ten rozpoczyna się od przepisania znanych informacji z ‘MoveInfo’ do ‘NotationInfo’. W następnej kolejności zapisywana jest notacja FEN, w sposób zbliżony do jej odczytu. Algorytm ten nie przedstawia żadnych interesujących rozwiązań, więc zostanie pominięty. Dwie następujące aktywności „Sprawdź czy zachodzi koniec gry” oraz „Zapisz notację SAN ruchu” posiadają bardziej złożoną logikę i zostaną przedstawione w szczególe.

III.3.3. Funkcja sprawdzająca wystąpienie zakończenia gry



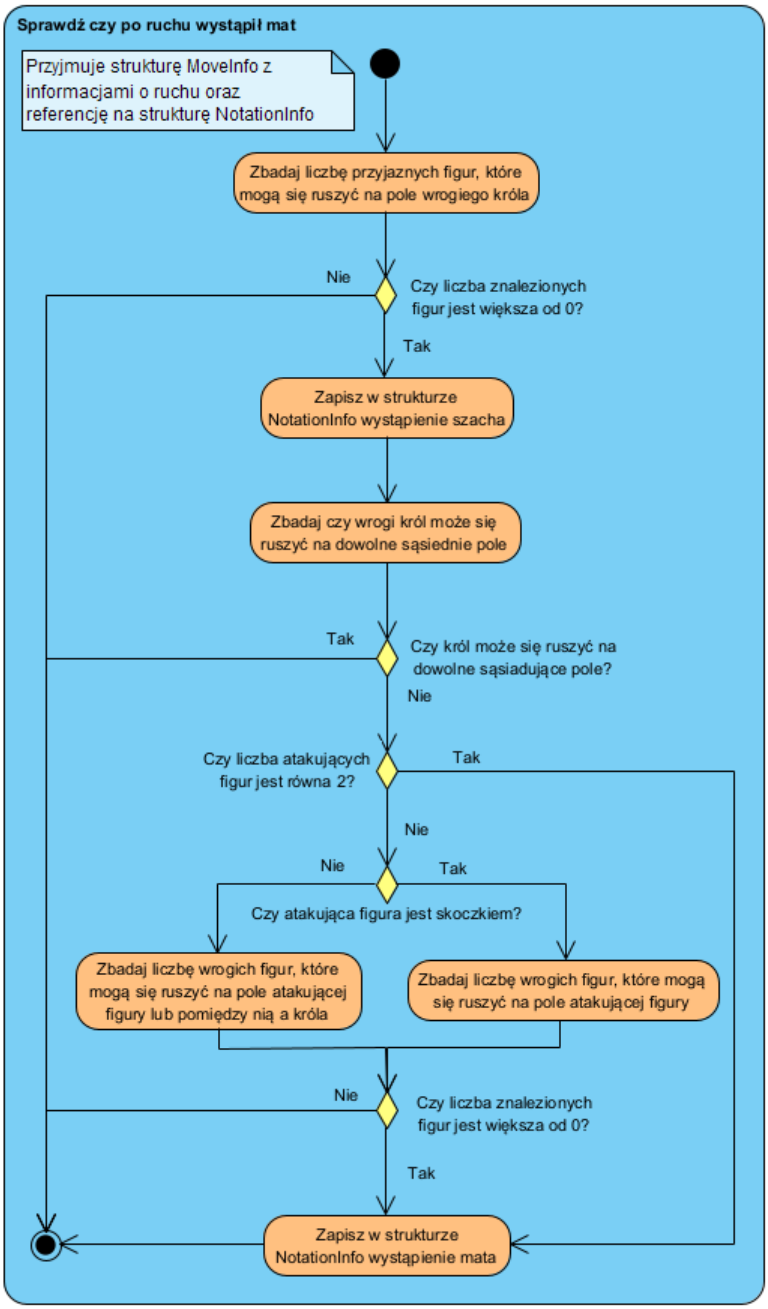
Rys. 17. Diagram aktywności funkcji sprawdzającej wystąpienie zakończenia gry

Źródło: Opracowanie własne

Pierwszym interesującym algorytmicznie etapem procesu określania rezultatu ruchu jest funkcja sprawdzająca występowanie i typ zakończenia partii. Przebieg tej funkcji został przedstawiony na diagramie aktywności (Rys. 17) i rozpoczyna się od zasymulowaniu ruchu gracza na planszy. Warto zauważyć, że działanie kodu jest zoptymalizowane w sposób przedstawiony na diagramie, czyli po wykryciu jednego sposobu zakończenia, algorytm natychmiast kończy działanie w celu zmniejszenia ilości wykonywanych obliczeń. Uwagę może też zwrócić to, że funkcja nie zwraca żadnych wartości. Wiąże się to z faktem, że jako jej argument przekazywana jest referencja na strukturę ‘NotationInfo’, której dane aktualizowane są w trakcie działania algorytmu, na podstawie rezultatów poszczególnych testów. Id gry przekazywane funkcji (jak można zauważyć w notatce w lewym górnym rogu diagramu) wykorzystywane jest w ostatniej aktywności „Sprawdź czy po ruchu wystąpił remis poprzez trzykrotne

powtórzenie pozycji”, co zostanie przedstawione na koniec omawiania pozostałych aktywności.

III.3.4. Funkcja sprawdzająca wystąpienie szach-mata



Rys. 18. Diagram aktywności funkcji sprawdzającej wystąpienia mata

Źródło: Opracowanie własne

Działanie funkcji sprawdzającej występowanie mata przedstawiono na diagramie (Rys. 18). W pierwszej aktywności wykorzystywana jest przedstawiona wcześniej funkcja (Kod. 4) do zliczania figur, które mogą ruszyć się na zadane pole, w celu znalezienia bierek, które po ruchu gracza atakują wrogiego króla. Jeśli algorytm nie zwróci żadnych wyników, to funkcja kończy działanie bez zapisania występowania

mata. W przeciwnym wypadku, w pierwszej kolejności zapisywane jest wystąpienia szacha, co jest ważną informacją dla notacji SAN. W drugiej kolejności, sprawdzane jest czy wrogi król może się ruszyć na dowolne sąsiadujące, niezagrożone atakami pole. Jeśli tak, to znowu mat nie jest zapisywany, a funkcja kończy działanie, jeśli nie, to w zależności od tego ile wcześniej znaleziono atakujących figur, podejmowane są różne działania. Jeśli znaleziono dwie atakujące figury, to znaczy, że wystąpił tak zwany „podwójny szach", którego jedyną obroną jest przesunięcie króla. Skoro we wcześniejszym etapie algorytm stwierdził, że wrogi król nie posiada legalnych ruchów, to znaczy że zachodzi mat, który jest zapisywany i funkcja kończy działanie. Jeśli na początku algorytmu wykryto tylko jedną atakującą figurę, to wrogi król ma dodatkowo możliwość obronienia się przed atakiem poprzez zasłonięcie się figurą odpowiadającą jego kolorowi, bądź zbicie atakującej bierki. Obie te możliwości sprawdzane są kodem (Kod. 4), tym razem jednak uwzględnia on bezpieczeństwo króla po ruchu, gdyż ochronienie króla przed atakiem jest nieskuteczne, jeśli odsłania go to na inne zagrożenia. Końcowo, gdy żadna metoda obrony wrogiego króla nie zadziała, podobnie jak poprzednio, stwierdzany i zapisywany w 'NotationInfo' jest mat. Przypadek atakowania króla więcej niż dwoma bierkami na raz nie może wystąpić w klasycznych szachach, więc nie jest obsługiwany.

III.3.5. Funkcja sprawdzająca wystąpienie remisu z zasady pięćdziesięciu ruchów

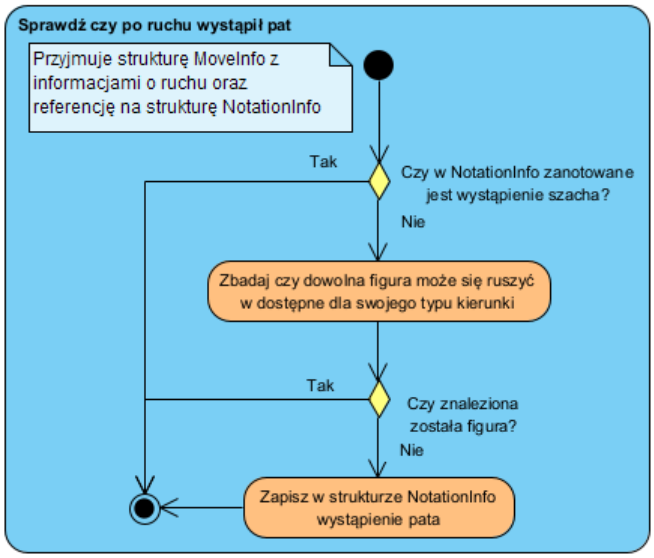
W przypadku gdy system nie wykryje mata, sprawdzane jest występowanie remisów, poczynając od zasady pięćdziesięciu ruchów. Ten rodzaj zakończenia ma miejsce, gdy obaj gracze wykonają pięćdziesiąt ruchów z rzędu bez poruszenia się pionkiem i bez bicia wrogich figur. Informacja o takich ruchach jest częścią notacji FEN, więc zaprezentowany kod funkcji (Kod. 8) w bardzo prosty sposób na podstawie tej danej może określić wystąpienie tego typu remisu. Oczywiście, w celu zapewnienia poprawnego działania funkcji, zadbano o poprawne resetowanie licznika półruchów w notacji FEN, kiedy gracze wykonają akcje inne niż wyżej wymienione.

Kod. 8. Funkcja sprawdzająca wystąpienie remisu z zasady pięćdziesięciu ruchów

```
1   def retrieve_fifty_move_rule_information(notation_info,
2   move_info):
3       if move_info.half_move_clock >= 100:
4           notation_info.draw_type = DrawType.FiftyMoveRule
```

Źródło: Opracowanie własne

III.3.6. Funkcja sprawdzająca wystąpienie pata

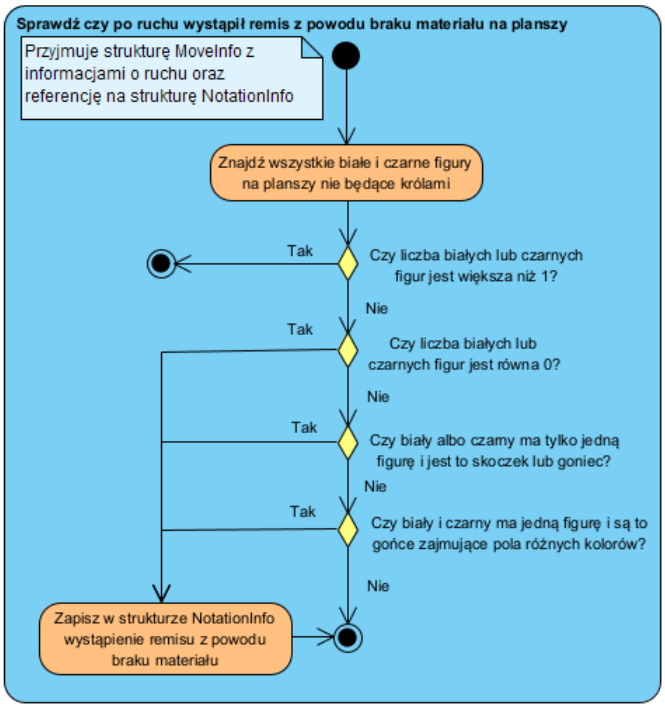


Rys. 19. Diagram aktywności funkcji sprawdzającej wystąpienia pata

Źródło: Opracowanie własne

Pat będący drugim rodzajem remisu sprawdzanym przez funkcję badającą zakończenie rozgrywki występuje, gdy żaden z graczy nie może wykonać legalnego ruchu. Przebieg algorytmu sprawdzającego wystąpienie tego remisu widoczny jest na diagramie aktywności (Rys. 19). W pierwszym kroku, sprawdzane jest wystąpienie szacha w ‘NotationInfo’, które potencjalnie mogło zostać zapisane w poprzednio omawianej funkcji sprawdzającej zajście mata. Jeśli szach został wcześniej stwierdzony i program przeszedł do fazy sprawdzania remisów, to można stwierdzić, że wrogi król ma możliwość obronienia się przed matem (może wykonać legalny ruch), więc pat nie zachodzi. Jeśli szach nie występuje, algorytm przechodzi do następnej fazy i dla każdej figury na planszy o kolorze odpowiadającemu następnej turze, sprawdzana jest możliwość wykonania przez nią ruchu w dowolnym kierunku odpowiadającemu jej typowi. Przy znalezieniu pierwszej instancji takiej bierki program przerywa pracę nie stwierdziwszy pata, jeśli jednak algorytm przeszuka całą planszę i nie znajdzie żadnego wyniku, to kończy pracę zapisując wystąpienie tego remisu. Należy zwrócić uwagę, że do przeszukiwania ruchów omawiana funkcja korzysta z własnej oryginalnej logiki, ponieważ rozpatruje możliwość poruszenia się każdej figury jedynie na najbliższe dostępne pola, zdefiniowane w specjalnej tablicy zawartej w funkcji. Przykładowo, jeśli zostanie znaleziona wieża, sprawdzana jest walidacja (wraz z bezpieczeństwem króla po ruchu), dla następujących ruchów: ‘(0, 1)’, ‘(0, -1)’, ‘(1, 0)’, ‘(-1, 0)’ - gdzie pierwsza liczba określa przesunięcie wskutek ruchu w osi poziomej, a druga w osi pionowej.

III.3.7. Funkcja sprawdzająca wystąpienie remisu z powodu braku materiału



Rys. 20. Diagram aktywności funkcji sprawdzającej wystąpienie remisu przez brak materiału

Źródło: Opracowanie własne

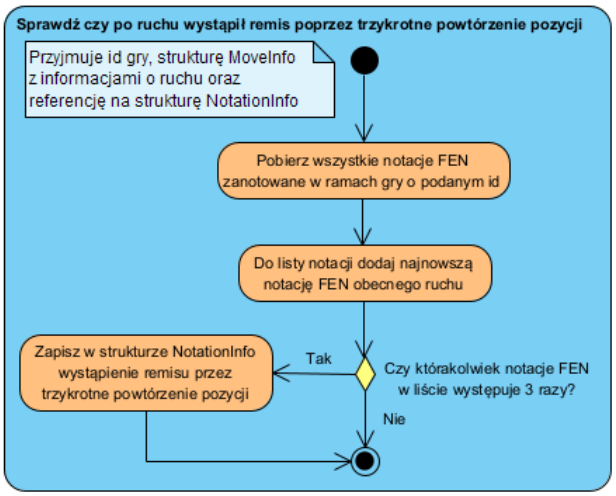
Proces sprawdzania remisu poprzez brak materiału na planszy przedstawiono na diagramie aktywności (Rys. 20). Ten typ zakończenia partii zachodzi w specyficznie zdefiniowanych sytuacjach, gdy żaden z graczy nie posiada figur wystarczających do zmatowania wrogiego króla. Szczególne przypadki rozpatrywane przez przedstawiony algorytm zaprezentowano w postaci tabeli (Tab. 3).

Tab. 3. Przypadki wystąpienia remisu przez brak materiału

Lp.	Typ remisu	Warunek
1	Sami królowie	Na planszy pozostają wyłącznie królowie (liczba białych i czarnych figur równa 0).
2	Królowie i goniec/skoczek	Jeden z graczy ma króla i dokładnie jedną figurę (goniec lub skoczek).
3	Królowie i dwa gońce przeciwnych kolorów oraz na różnych kolorach pól	Obaj gracze mają po jednym królu i jednym gońcu, przy czym gońce są na polach o przeciwnych kolorach.

Źródło: Opracowanie własne

III.3.8. Funkcja sprawdzająca wystąpienie remisu przez trzykrotne powtórzenie pozycji



Rys. 21. Diagram aktywności funkcji sprawdzającej wystąpienie remisu przez trzykrotne powtórzenie pozycji

Źródło: Opracowanie własne

Funkcja sprawdzająca remis przez trzykrotne powtórzenie pozycji, której logika zaprezentowana jest na diagramie aktywności (Rys. 21) działa poprzez pobranie z bazy danych wszystkich notacji FEN zapisanych w ramach gry i sprawdzenia czy któraś z nich z najnowszą włącznie wystąpiła trzy razy, co świadczy o tym typie remisu.

III.3.9. Funkcja zapisująca notację SAN

Ostatnim etapem algorytmu rozstrzygającego rezultat ruchu jest generowanie notacji SAN, będącej powszechnym sposobem zapisu ruchu w szachach, zawierającym informację o typie ruszanej figury, polu docelowym i akcjach ruchu, takich jak bicie, szach albo promocja pionka.

Przykładem notacji SAN może być ruch skoczkiem na pole f3, zapisany jako „Nf3”. Jeśli w trakcie ruchu dochodzi do bicia, np. na polu e5, notacja zostaje rozszerzona do „Nxe5”. Gdyby ruch dodatkowo skutkował atakiem króla, na końcu notacji pojawiłby się ‘+’, co dałoby notację „Nxe5+”. W ramach notacji SAN można wyróżnić jeszcze kilka przypadków brzegowych takich jak promocja pionka (na przykład „e8=Q”) albo roszada („O-O” albo „O-O-O” w zależności od kierunku roszady).

Dzięki utworzonej i wypełnionej wcześniej zgromadzonymi informacjami strukturze ‘NotationInfo’ zapis notacji SAN jest prosty i polega na wykonaniu szeregu instrukcji warunkowych, przekładających cechy ruchu na łańcuchy znaków. Wyjątkiem od zasady jest rzadki problem, występujący gdy więcej niż jedna bierka tego samego typu może wykonać ruch na to samo pole docelowe. W tej sytuacji należy

ujednoznacznić ruszaną bierkę wskazując rząd i/lub kolumnę pola źródłowego ruchu, w zależności od tego które z nich są współdzielone.

Kod. 9. Funkcja ujednoznaczniająca pole poruszanej figury

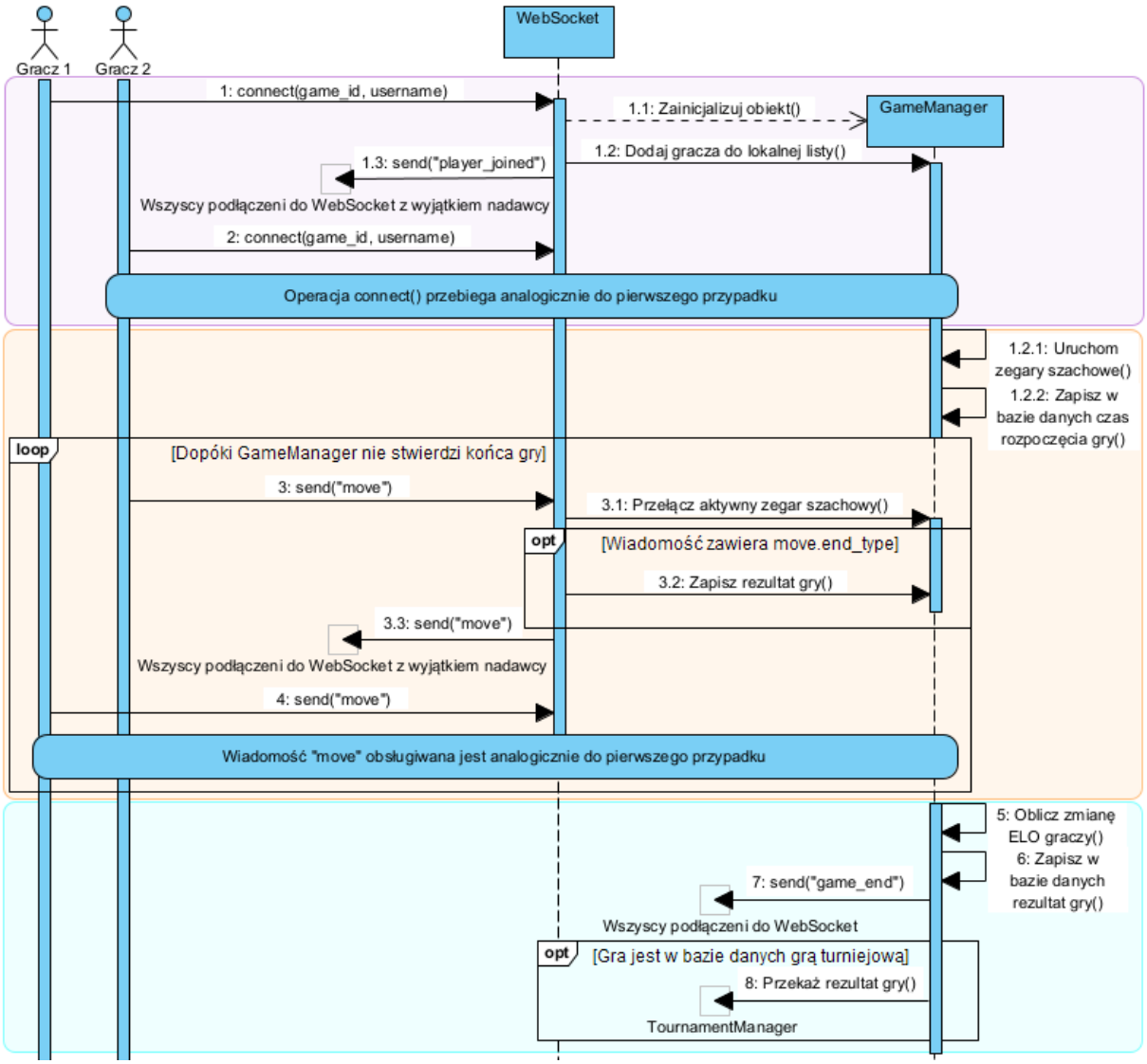
```
1   def get_move_disambiguation(move_info):
2       from .move_validation_algorithms import
    get_pieces_that_can_go_to_square
3       ambiguous_pieces =
    get_pieces_that_can_go_to_square(move_info.target_x,
    move_info.target_y, move_info, move_info.white_turn, True)
4
5       ambiguous_pieces = [
6           p for p in ambiguous_pieces
7           if move_info.chess_board[p[0]][p[1]].type ==
    move_info.source_piece.type
8           and (p[0], p[1]) != (move_info.source_x,
    move_info.source_y)
9       ]
10
11      if len(ambiguous_pieces) == 0:
12          return ''
13
14      is_column_ambiguous = any(p[0] == move_info.source_x for
    p in ambiguous_pieces)
15      is_row_ambiguous = any(p[1] == move_info.source_y for p
    in ambiguous_pieces)
16
17      if is_column_ambiguous and is_row_ambiguous:
18          return convert_to_algebraic(move_info.source_x,
    move_info.source_y)
19      elif is_column_ambiguous:
20          return convert_to_algebraic(None, move_info.source_y)
21      else:
22          return convert_to_algebraic(move_info.source_x, None)
```

Źródło: Opracowanie własne

Problem ujednoznaczniania ruszanej bierki rozwiązany został w funkcji (Kod. 9). Przedstawiony algorytm najpierw za pomocą wcześniej omawianego kodu (Kod. 4) pobiera listę wszystkich figur koloru ruszanej figury, które mogą przejść na pole docelowe z uwzględnieniem bezpieczeństwa króla po ruchu. Lista ta jest następnie filtrowana, tak aby zawierała jedynie bierki tego samego typu co ruszana figura. Na koniec na podstawie współdzielenia tej samej kolumny bądź rzędu planszy, zapisywana jest odpowiednia informacja o polu źródłowym figury ruszanej przez gracza.

III.4. Klasa zarządzająca przebiegiem gry szachowej

Klasa ‘GameManager’ zaimplementowana w backendzie systemu odpowiada za zarządzanie przebiegiem gry szachowej. We współpracy z gniazdem sieciowym zajmuje się wykonywaniem różnorodnych czynności, od aktualizowania stanu gry, po zarządzanie zegarami szachowymi i specjalnymi akcjami graczy, takimi jak cofanie ruchu. Komunikacja między użytkownikami, a gniazdami sieciowymi i klasą ‘GameManager’ w trakcie rozgrywki szachowej została w uproszczony sposób przedstawiona na diagramie sekwencji (Rys. 22) z rozdzieleniem na kluczowe etapy rozgrywki według kolorów. Etapy określono na podstawie tabeli (Tab. 1), można więc wyróżnić stan oczekiwania na rozpoczęcie (purpurowy), rozgrywania partii (pomarańczowy) oraz zakańczania gry (niebieski).



Rys. 22. Diagram sekwencji cyklu życia klasy zarządzającej przebiegiem gry szachowej

Źródło: Opracowanie własne

III.4.1. Stan oczekiwania gry szachowej na rozpoczęcie

Pierwszą rzeczą jaką można zauważyć na diagramie w purpurowej sekcji, jest to że obiekt klasy ‘GameManager’ tworzony jest automatycznie po otrzymaniu przez gniazdo sieciowe pierwszej wiadomości ‘connect’. Skutkuje to natychmiastowym pobraniem przez klasę informacji o grze z bazy danych oraz inicjalizację zmiennych, głównie odpowiedzialnych za ustawienia zegarów szachowych. Ponieważ wiadomości gniazd sieciowych prawie zawsze przekazywane są do innych użytkowników w celu dynamicznego aktualizowania interfejsów, na diagramie użyto bram (kwadraty pomiędzy liniami życia), które obrazują do jakiej grupy odbiorców przesyłana jest wiadomość. W celu przedstawienia analogicznie wykonywanych operacji przy dołączaniu drugiego użytkownika do gry, użyto elementu kontynuacji (niebieska pozioma linia rozciągająca się między liniami życia). Jedynym wyjątkiem od analogicznego działania w tym przypadku jest to, że przy dołączaniu drugiego gracza obiekt klasy ‘GameManager’ nie jest ponownie inicjalizowany, gdyż już istnieje.

III.4.2. Stan rozgrywania partii szachowej

Po dołączeniu się drugiego gracza, klasa ‘GameManager’ zmienia stan rozgrywki na rozpoczęty (pomarańczowa sekcja diagramu) poprzez aktualizacje odpowiednich pól w bazie danych i startuje zegary szachowe. ‘GameManager’ nie wysyła żadnych wiadomości oznajmiających rozpoczęcie gry, bo po stronie frontendu ta akcja rozpoznawana jest automatycznie na podstawie wiadomości dołączania się użytkowników. W trakcie trwania gry użytkownicy na przemian wykonują ruchy, które dopiero po udanej walidacji przesyłane są w postaci wiadomości, w celu aktualizowania stanu planszy u wszystkich osób na stronie gry. Wykonanie ruchu powoduje też zwiększenie inkrementu czasowego u tego gracza przez klasę ‘GameManager’ oraz zmianę aktywnie odliczającego zegara na zegar drugiego zawodnika. W trakcie gry zegary szachowe automatycznie co sekundę wysyłają aktualizację swoich stanów poprzez wiadomość ‘clock_update’, co zostało pominięte na diagramie ze względu na trudność przedstawienia tego zachowania z powodu jego współbieżnego charakteru. Warto zauważyć, że pętla opisująca na diagramie fazę rozgrywania partii szachowej może zostać przerwana z licznych powodów. Oprócz przedstawionego na ilustracji powodu zakończenia gry poprzez zawarcie informacji ‘move.end_type’ w wiadomości ruchu, partia może zostać zakończona przez klasę ‘GameManager’ wskutek takich sytuacji jak wyjście jednego z graczy z partii, ustalenie przez graczy remisu, poddania się jednego z zawodników czy awarii silnika szachowego (niepoprawnych sugestii ruchów trzy razy z rzędu). Wszystkie te akcje oznajmiane są tak jak wcześniej wysyłaniem wiadomości przez gniazda sieciowe przez jednego z graczy.

III.4.3. Stan zakańczania partii szachowej

Po zakończeniu partii ‘GameManager’ przechodzi do etapu finalizowania gry przedstawionego kolorem niebieskim na diagramie. Zadaniami tej klasy jest obliczenie

zmiany rankingu ELO u obu graczy oraz zapisanie rezultatów do bazy danych. System punktacji ELO liczony jest na podstawie wyników rozgrywek, różnicy umiejętności między przeciwnikami oraz indywidualnej liczby gier rozegranych na platformie przez każdego z nich.

Kod. 10. Funkcja obliczająca zmianę rankingu ELO gracza po zakończeniu partii

```
1       def calculate_k_factor(games_played, base_k=128,
    min_k=32):
2           return max(min_k, base_k - games_played * 4)
3
4       def calculate_elo_change(player_elo, opponent_elo, score,
    k_factor):
5           expected_score = 1 / (1 + 10 ** ((opponent_elo -
    player_elo) / 400))
7           return round(k_factor * (score - expected_score))
```

Źródło: Opracowanie własne

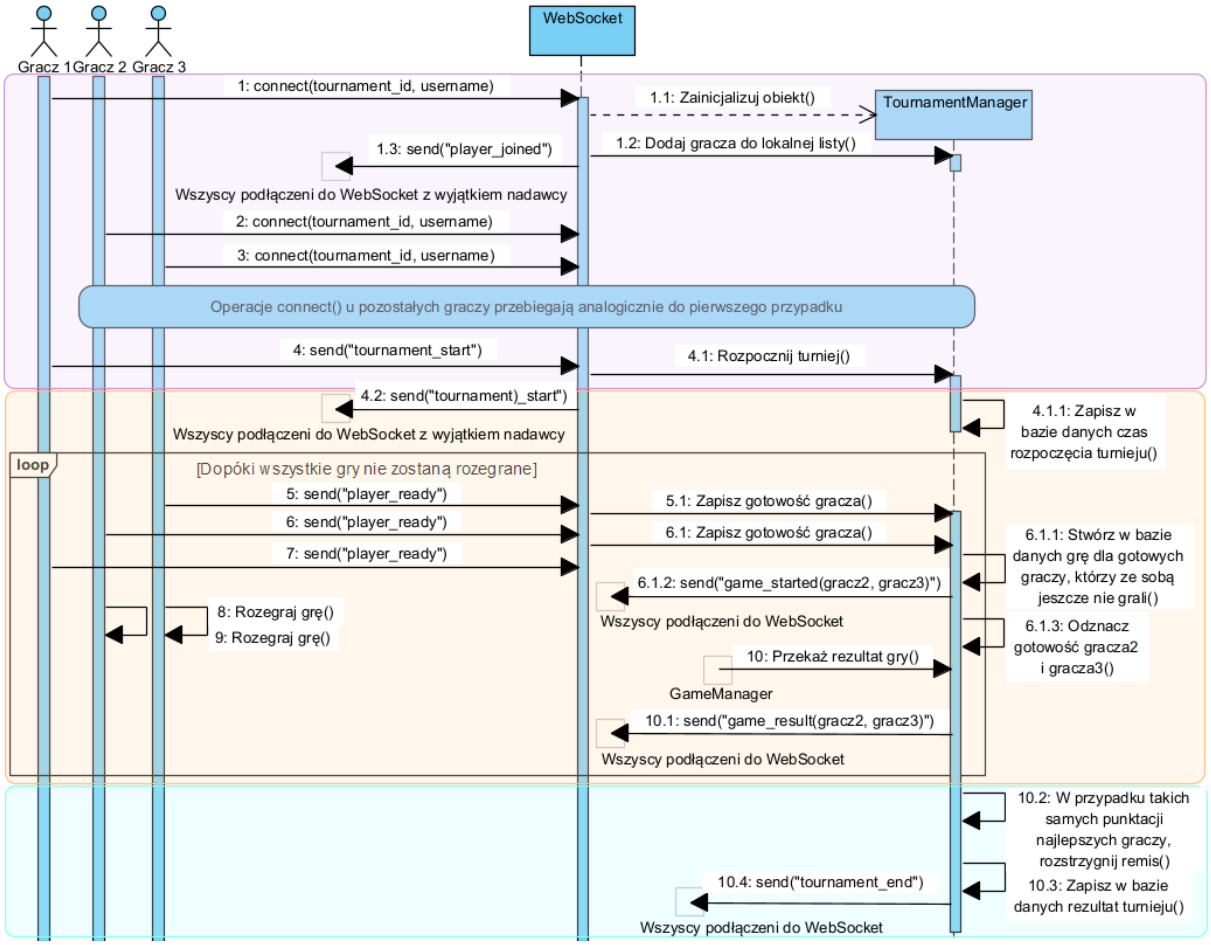
Kod (Kod. 10) pokazuje dwie najważniejsze funkcje klasy ‘GameManager’, odpowiedzialne za poprawne liczenie rankingu według powszechnie przyjętej w świecie szachów formuły. Funkcja ‘calculate_k_factor’ odpowiedzialna jest za obliczanie ‘współczynnika k’, czyli zmiennej która jest tym większa, im mniej gier rozegrano na koncie użytkownika. W rezultacie, zmienna ta powoduje, że pierwsze gry na koncie mają największy wpływ na ranking gracza. Funkcja ‘calculate_elo_change’ przechowuje w sobie sedno algorytmu ELO, który za pomocą formuły matematycznej oraz cech gry i zawodników zwraca zmianę tego rankingu dla każdego z nich. Po aktualizacji informacji w bazie danych, wysyłana jest wiadomość ‘game_end’, która jest ostatnim sygnałem dla użytkownika platformy do odświeżenia interfejsów w celu zobaczenia rezultatów.

W przypadku gdy ‘GameManager’ wykryje, że rozegrana partia jest częścią rozgrywek turniejowych, dodatkowo pobiera z bazy danych id odpowiedniego turnieju i przesyła do klasy sterującej jego przebiegiem informację z rezultatem zakończonej gry szachowej.

III.5. Klasa zarządzająca przebiegiem turnieju szachowego

W ramach zarządzania przebiegiem turnieju, w backendzie systemu zaimplementowano klasę ‘TournamentManager’. Do jej podstawowych zadań wlicza się dobieranie graczy w pary w celu rozgrywania indywidualnych partii szachowych, zarządzanie punktacją zawodników turnieju, rozstrzyganie zwycięzcy po rozegraniu wszystkich gier i zapisywanie na bieżąco nowych informacji do bazy danych. Podobnie jak dla klasy zarządzającej przebiegiem gry szachowej, dla klasy ‘TournamentManager’ utworzono diagram sekwencji (Rys. 23) w uproszczony sposób obrazujący cykl życia

omawianej klasy, w trakcie którego we współpracy z gniazdami sieciowymi odbierającymi wiadomości od zawodników wykonywane są odpowiednie operacje. Podobnie jak na poprzednim diagramie, zastosowano mniej standardowe elementy bram i kontynuacji w celu przedstawienia odpowiednio komunikacji między obiektami lub grupami obiektów niewchodzących w skład diagramu oraz operacji wykonywanych analogicznie do zachodzących wcześniej. Kolorami oznaczono też główne stany turnieju podobnie jak poprzednio, czyli na purpurowo oznaczony jest stan oczekiwania na rozpoczęcie, na pomarańczowo stan rozgrywania turnieju, a na niebiesko stan finalizacji turnieju w systemie.



Rys. 23. Diagram sekwencji cyklu życia klasy zarządzającej przebiegiem turnieju szachowego

Źródło: Opracowanie własne

III.5.1. Stan oczekiwania turnieju na rozpoczęcie

W stanie oczekiwania turnieju na rozpoczęcie (purpurowy na diagramie), podobnie jak dla klasy zarządzającej grą, obiekt klasy ‘TournamentManager’ tworzony jest po otrzymaniu pierwszej wiadomości ‘connect’ przez gniazdo sieciowe. Natychmiast po utworzeniu tej klasy inicjalizuje ona potrzebne zmienne, w większości na podstawie danych turnieju pobranych z bazy danych. Szczególnie ważne jest, że ‘TournamentManager’ zapisuje dołączających się graczy (w tym

pierwszego) w lokalnej liście, na podstawie której tworzy i aktualizuje rekordy tabeli uczestników turnieju w bazie danych.

III.5.2. Stan rozgrywania turnieju szachowego

Po dołączeniu się wszystkich zawodników i rozpoczęciu turnieju przez jego właściciela poprzez wysłanie wiadomości ‘tournament_start’, turniej przechodzi do stanu rozgrywania (pomarańczowy na diagramie), a klasa ‘TournamentManager’ aktualizuje stan turnieju w bazie danych i rozpoczyna proces dobierania graczy w pary. Proces ten polega w dużej mierze na wiadomościach wysyłanych przez graczy, oznajmiających ich gotowość (‘player_ready’), co powododuje dodanie ich do lokalnej listy gotowych graczy w klasie ‘TournamentManager’. Przy każdej aktualizacji gotowości, omawiana klasa sprawdza, czy istnieją inni gotowi gracze, którzy w ramach turnieju w bazie danych nie rozgrywali gry z nowo zgłoszonym. W przypadku znalezienia gotowego przeciwnika, ‘TournamentManager’ automatycznie tworzy w bazie danych turniejową grę szachową i przydziela do niej gotowych zawodników, a następnie za pomocą wiadomości ‘game_started’ z odpowiednim parametrami, wywołuje w przeglądarkach graczy sygnał przenoszący ich na stronę wspomnianej gry. Po zakończeniu partii szachowej, klasa zarządzająca jej przebiegiem w systemie przekazuje klasie ‘TournamentManager’ jej wynik, na podstawie którego aktualizuje ona punktacje lokalnie i w bazie danych, po czym wysyłana jest wiadomość ‘game_result’, będąca sygnałem do odświeżenia interfejsów po stronie użytkownika, w celu zobaczenia najnowszych punktacji. Dodatkowo, ‘TournamentManager’ po zakończeniu każdej gry sprawdza na podstawie informacji w bazie danych, czy liczba gier rozegranych w ramach turnieju równa jest liczbie kombinacji wszystkich par uczestników, jeśli tak, to przechodzi do ostatniej fazy zakańczania turnieju. Przed przejściem do tego etapu, warto jeszcze wspomnieć, że w przypadku rozłączenia się dowolnego użytkownika z turnieju w trakcie jego trwania, klasa zarządzająca przebiegiem turnieju pobiera wszystkie gry turniejowe z jego uczestnictwem i na ich podstawie usuwa z punktacji innych graczy rezultaty tych gier. Same gry zawodnika, który opuścił turniej również usuwane są z bazy danych po wykonaniu poprzednich operacji.

III.5.3. Stan zakańczania turnieju szachowego

Proces finalizacji oznaczony kolorem niebieskim na diagramie polega na zapisaniu nazwy zwycięzcy oraz czasu zakończenia w bazie danych, a następnie wysłaniu wiadomości ‘tournament_end’ w celu aktualizacji interfejsów u użytkowników. W przypadku remisu klasa ‘TournamentManager’ implementuje hierarchię metod rozstrzygania wyniku aż do momentu wyłonienia zwycięzcy, są to:

- Metoda Sonneborn-Bergera, polegająca na sumowaniu punktów przeciwników pokonanych przez gracza.
- Metoda wyboru gracza z najwyższą liczbą zwycięstw

- Metoda wyboru gracza z najwyższym rankingiem ELO
- Metoda losowego wyboru gracza

Wszystkie wymienione metody są proste w implementacji i bazują ściśle na danych rozegranych gier oraz zawodników turnieju pobieranych z bazy danych.

III.6. Przykładowa implementacja serwera z silnikiem szachowym

Przykładowy serwer z silnikiem szachowym został zaimplementowany w technologii Django i pełni rolę wzorcowego rozwiązania dla użytkowników platformy, którzy mogą tworzyć własne serwery i udostępniać je za pomocą adresów URL. Serwer komunikuje się lokalnie z silnikiem szachowym Stockfish, jednym z najpopularniejszych silników dostępnych na licencji wolnego oprogramowania. Jego celem jest zapewnienie prostego interfejsu API, który umożliwia integrację z systemami zewnętrznymi, w tym z platformą wspierającą rozgrywki szachowe będącą głównym tematem pracy. Działanie serwera opiera się na zestawie punktów końcowych REST API, opracowanych zgodnie z najpowszechniejszym protokołem komunikacji interfejs-silnik, czyli protokołem UCI:

- ‘/ucinewgame’ – rozpoczyna nową partię w silniku szachowym poprzez komendę ucinewgame.
- ‘/isready’ – sprawdza, czy silnik jest gotowy do przetwarzania kolejnych komend, oraz w razie potrzeby przeprowadza jego restart.
- ‘/position’ – ustawia podaną pozycję w notacji FEN w silniku szachowym, co pozwala na analizę konkretnych sytuacji na planszy.
- /’go’ – wywołuje algorytm obliczający najlepszy ruch dla aktualnej pozycji w silniku i zwraca jego wynik w postaci notacji koordynatowej.
- ‘/stop’ – zatrzymuje aktualne obliczenia w silniku, co może być użyteczne w przypadku zmiany kontekstu gry.
- ‘/quit’ – kończy działanie silnika poprzez bezpieczne zamknięcie procesu.
- ‘/chat’ – umożliwia wysyłanie dowolnych komend do silnika oraz odbieranie jego odpowiedzi w celach diagnostycznych.

Szczegółowy opis protokołu można znaleźć na stronie autorów silnika Stockfish [13].

Komunikacja ze Stockfishem została zrealizowana poprzez uruchomienie go przy starcie serwera jako proces za pomocą komendy ‘subprocess.Popen’. Wykorzystanie ‘stdin’, ‘stdout’ oraz ‘stderr’ jako potoków (ang. *pipe*) umożliwia dwukierunkową wymianę danych między serwerem a silnikiem szachowym. Uruchamianie silnika szachowego jako niezależny podproces, pozwala na wysyłanie do niego komend oraz odbieranie odpowiedzi w sposób asynchroniczny. Dane wyjściowe generowane przez silnik również odczytywane są w osobnym wątku. Wątek ten działa w połączeniu z mechanizmem kolejek (ang. *queue*), co pozwala na przechowywanie i bezpieczne przesyłanie wyników przetwarzania do głównego serwera. Dzięki takiej konfiguracji wyniki pracy silnika mogą być współdzielone pomiędzy punktami

końcowymi. Definicje i inicjalizację opisanych mechanizmów przedstawiono w kodzie (Kod. 11).

Kod. 11. Inicjalizacja silnika szachowego oraz kolejki odczytu jego odpowiedzi

```
process = subprocess.Popen(
    ['/app/stockfish/stockfish'],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    universal_newlines=True,
    bufsize=1,
)

output_queue = Queue()

def read_engine_output(process):
    while True:
        line = process.stdout.readline().strip()
        if line:
            output_queue.put(line)

engine_thread = Thread(target=read_engine_output,
args=(process,))
engine_thread.daemon = True
engine_thread.start()
```

Źródło: Opracowanie własne

Punkty końcowe REST API serwera z silnikiem szachowym zostały zaimplementowane z użyciem dekoratora @api_view, co pozwala na prostą obsługę różnych metod HTTP, takich jak POST czy GET. Przykładowy punkt końcowy ‘/go’ (Kod. 12) ilustruje sposób komunikacji z silnikiem Stockfish, realizując zadanie wyznaczenia najlepszego ruchu dla bieżącej pozycji.

Kod. 12. Przykład punktu końcowego ‘/go’ serwera z silnikiem szachowym

```
@api_view(['POST'])
def go(request):
    parameters = request.data.get('parameters', '')

    process.stdin.write('go ' + parameters + '\n')
    process.stdin.flush()

    response = ''
    while True:
        line = output_queue.get()
        response += line + '\n'
        if 'bestmove' in line:
            break

    return HttpResponse(response)
```

Źródło: Opracowanie własne

Działanie kodu rozpoczyna się od odczytu danych z ciała żądania HTTP, które przekazywane są jako parametr ‘request’. Klucz ‘parameters’ zawarty w danych wejściowych pozwala na dynamiczne określenie opcji wysyłanych do silnika, takich jak limity czasowe lub inne szczegóły obliczeń. Dane te są następnie formatowane jako ciąg znaków i przesyłane do silnika za pomocą strumienia wejściowego procesu Stockfish, przy użyciu funkcji ‘write()’. Wyniki przetwarzania przez silnik są asynchronicznie odczytywane z kolejki ‘output_queue’, do której wątek odczytu danych dodaje kolejne wiersze odpowiedzi. Mechanizm ten umożliwia bieżące monitorowanie stanu obliczeń bez blokowania działania serwera. Odpowiedzi są przechowywane w zmiennej ‘response’, która jest iteracyjnie uzupełniana do momentu, aż wiersz charakterystyczny dla zakończenia operacji (‘bestmove’) zostanie napotkany. Po wykryciu tego elementu pętla przetwarzania zostaje przerwana, a cała odpowiedź zwracana jest w formie odpowiedzi HTTP.

III.7. Testy zaimplementowanych funkcji systemu

Podczas tworzenia strony internetowej wszystkie funkcje były testowane ręcznie, co pozwoliło na szybkie identyfikowanie błędów oraz weryfikację działania systemu w różnych scenariuszach. Testy funkcji systemu obejmowały uruchamianie poszczególnych modułów aplikacji i sprawdzanie ich zgodności z założeniami projektowymi. Na przykład, po implementacji systemu zarządzania grami testowano takie funkcje jak dołączanie graczy do gier, wykonywanie ruchów, aktualizacja zegarów, a także zapis wyników do bazy danych. Zwracano szczególną uwagę na przypadki brzegowe, takie jak nieprawidłowe ruchy, rozłączenie z internetem czy wyjście jednego z graczy w trakcie trwania rozgrywki. Aby dokładniej przetestować

mechanizmy komunikacji, przeprowadzano testy w wielu instancjach przeglądarki, logując się do aplikacji jako gość w różnych oknach. Metoda ta zapobiegała współdzieleniu sesji i pozwalała na realistyczne symulacje interakcji między użytkownikami.

Oprócz podstawowego testowania funkcji systemu w przeglądarkach Google Chrome oraz Microsoft Edge, wykorzystano narzędzia deweloperskie przeglądarek do analizy poprawności działania aplikacji. W zakładkach Console, Application i Network sprawdzano, czy wszystkie zasoby aplikacji (takie jak skrypty, style CSS i obrazy) są poprawnie ładowane, a dane przesyłane między frontendem i backendem zgodne z oczekiwaniami. Szczególną uwagę poświęcono także weryfikacji odpowiedzi API oraz zgodności danych przesyłanych do bazy danych.

Poprawność zapisu i odczytu danych z bazy była testowana za pomocą pgAdmin, co umożliwiło weryfikację działania mechanizmów backendu oraz poprawności operacji CRUD (Create, Read, Update, Delete). W procesie testowania aplikacji wykorzystano także narzędzie Docker, dzięki któremu na bieżąco monitorowano logi kontenerów backendu, frontendu oraz bazy danych. Pozwoliło to na szybkie wykrywanie potencjalnych problemów, takich jak błędy serwera czy niespójności w konfiguracji kontenerów.

Dodatkowo, przeprowadzono testy instalacji i działania aplikacji na różnych urządzeniach i konfiguracjach sprzętowych, co miało na celu upewnienie się, że aplikacja działa poprawnie w różnych środowiskach. Testowano między innymi urządzenia o różnej rozdzielczości ekranu oraz zróżnicowane pod względem wydajności procesory, aby zapewnić responsywność i poprawne działanie aplikacji niezależnie od platformy.

III.8. Kierunki rozwoju systemu

Jednym z najbardziej perspektywicznych kierunków rozwoju jest wprowadzenie różnych wariantów szachowych. Obecnie zaimplementowane struktury danych i algorytmy są wystarczająco elastyczne, aby obsługiwać szachy 960 (ang. *Fischer Random Chess*), anty-szachy, czy inne popularne odmiany gry. Rozszerzenie to mogłoby obejmować dostosowanie reguł ruchów figur, a także modyfikację sposobów ich początkowego rozmieszczenia. Dzięki modularnemu podejściu, zmiany te mogą być implementowane w sposób minimalnie inwazyjny, co pozwala na zachowanie spójności i stabilności istniejącego systemu.

Rozwój funkcji społecznościowych również stanowi ważny aspekt dalszego rozwoju systemu. Możliwość tworzenia list znajomych pozwoliłaby użytkownikom na zapraszanie się nawzajem do prywatnych gier, organizowanie wspólnych turniejów czy wymianę wiadomości. Wdrożenie tej funkcji wymagałoby zarówno wprowadzenia nowych struktur danych w bazie, jak i zaprojektowania interfejsów użytkownika wspierających zarządzanie kontaktami oraz interakcje społeczne.

Kolejnym istotnym kierunkiem jest rozbudowa obecnego systemu rankingowego. Klasyczny system ELO mógłby zostać wzbogacony o szczegółowe statystyki, takie jak liczba rozegranych partii, procent wygranych, średnia liczba ruchów w grach wygranych i przegranych, czy dane dotyczące czasu gry. Takie informacje mogłyby być prezentowane w profilach graczy, co zwiększyłoby ich zaangażowanie i motywację do aktywnego korzystania z systemu.

Równie ważnym elementem rozwoju systemu jest poprawa interfejsu graficznego. Dodanie bardziej zaawansowanych animacji ruchów figur, interaktywnej nauki szachów dla początkujących oraz możliwości personalizacji wyglądu planszy i figur mogłoby znacząco poprawić wrażenia użytkowników. Dodatkowo, rozszerzenie aplikacji o pełną responsywność na różnych urządzeniach oraz integrację z platformami mobilnymi otworzyłoby dostęp do nowych grup użytkowników, co zwiększyłoby popularność systemu.

Podsumowanie

Celem niniejszej pracy było zaprojektowanie, implementacja systemu umożliwiającego organizację gier i turniejów szachowych zarówno dla graczy komputerowych, jak i ludzkich. Realizacja tego celu wymagała utworzenia kompleksowej platformy internetowej, integrującej różnorodne komponenty, takie jak autorskie algorytmy szachowe, mechanizmy zarządzania grami oraz intuicyjny interfejs użytkownika. Proces tworzenia systemu obejmował szczegółową analizę wymagań, projektowanie i implementację różnych poziomów architektury, a także testowanie funkcji systemu na kolejnych etapach wytwarzania.

Opracowana platforma została z powodzeniem ukończona, dostarczając solidne podstawy do rozgrywania gier oraz organizacji turniejów. Co najważniejsze, zgodnie z założeniami wspiera rozwój amatorskich silników szachowych, umożliwiając ich testowanie oraz integrację w praktycznych scenariuszach, co stanowi ważny wkład w popularyzację programowania oraz teorii gier. Testy funkcji systemowych przeprowadzone na różnych przeglądarkach oraz środowiskach sprzętowych potwierdziły poprawność działania kluczowych modułów i wysoką niezawodność systemu.

Wytworzona aplikacja wyróżnia się zastosowaniem autorskich algorytmów szachowych i struktur danych, cechujących się łatwym do zrozumienia podejściem funkcyjnym i obiektowym. Oprócz zwartego i czytelnego kodu, sama architektura systemu charakteryzuje się modularnością, co znacząco ułatwia jego adaptację do nowych potrzeb oraz rozszerzenie funkcji systemu. Potencjalne kierunki dalszego rozwoju obejmują wprowadzenie nowych możliwości społecznościowych, obsługę wariantów szachowych oraz rozwój interfejsu mobilnego, co dodatkowo zwiększyłoby dostępność i atrakcyjność platformy.

Podsumowując, opracowana platforma stanowi solidny fundament zarówno dla entuzjastów szachowych, jak i programistów zainteresowanych tworzeniem oraz analizą algorytmów. Dzięki modularności, przystępności oraz szerokim możliwościom rozbudowy platforma posiada ogromny potencjał, by przy wdrożeniu do środowiska produkcyjnego stać się wartościowym narzędziem edukacyjnym oraz inspiracją dla przyszłych projektów.

Bibliografia

[1] Aravind S.: Integrating REST APIs in Single Page Applications Using Angular and TypeScript. International Journal on Recent and Innovation Trends in Computing and Communication, 11 (2), 262–279, 2023. Dostępne na: https://ijritcc.org/index.php/ijritcc/article/view/11118

[2] https://v17.angular.io/docs [dostęp: 13.08.2024].

[3] Vainikka J.: Full-Stack Web Development Using Django REST Framework and React. Bachelor's thesis, Metropolia University of Applied Sciences, 2018. Dostępne na: https://www.theseus.fi/bitstream/handle/10024/146578/joel_vainikka.pdf

[4] https://docs.djangoproject.com/en/5.1/ [dostęp: 14.08.2024].

[5] https://www.django-rest-framework.org/ [dostęp: 14.08.2024].

[6] Sathio S. A., Siddiqui I. F., Arain Q. A.: A Secure Software Specification Development Strategy for Enterprises: A Case Study Approach. International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 7 (1), 260–266, 2021. Dostępne na: https://www.researchgate.net/publication/349716187_A_Secure_Software_Specification_Development_Strategy_for_Enterprises_A_Case_Study_Approach

[7] https://code.visualstudio.com/docs [dostęp: 12.08.2024].

[8] https://www.pgadmin.org/docs/ [dostęp: 17.08.2024].

[9] https://docs.docker.com/ [dostęp: 12.08.2024].

[10] Levy D. N. L., Newborn M.: How Computers Play Chess. W. H. Freeman & Co., New York, 1991, ISBN: 0-7167-8239-1.

[11] Kazi A. S.: Tracking and Annotating a Chess Game. Bachelor's dissertation, University of Edinburgh, 2013. Dostępne na: https://homepages.inf.ed.ac.uk/rbf/DISSERTATIONS/ug4_20130500.pdf

[12] Iqbal A.: An Algorithm for Automatically Updating a Forsyth-Edwards Notation String Without an Array Board Representation. arXiv preprint, 2020, DOI: 10.48550/arXiv.2009.03193.

[13] https://official-stockfish.github.io/docs/stockfish-wiki/UCI-&-Commands.html [dostęp: 2024-12-28].

Spis rysunków

Rys. 1. Diagram przypadków użycia panelu gier użytkownika 5
Rys. 2. Diagram przypadków użycia panelu profilu użytkownika..... 6
Rys. 3. Diagram aktywności procesu aktywacji silnika szachowego na stronie 8
Rys. 4. Diagram związków encji bazy danych systemu..... 17
Rys. 5. Strona główna aplikacji..... 22
Rys. 6. Strona nierozpoczętego turnieju..... 24
Rys. 7. Strona zakończonego turnieju 25
Rys. 8. Strona gry szachowej..... 26
Rys. 9. Strona profilu użytkownika..... 27
Rys. 10. Strona zarządzania silnikami szachowymi..... 28
Rys. 11. Strona informacyjna 29
Rys. 12. Diagram aktywności głównej gałęzi algorytmu walidacji ruchu szachowego. 32
Rys. 13. Przykład notacji FEN 33
Rys. 14. Diagram aktywności funkcji walidacji ruchu szachowego 35
Rys. 15. Diagram aktywności funkcji walidacji bezpieczeństwa króla po ruchu 37
Rys. 16. Diagram aktywności funkcji obliczania rezultatu ruchu..... 41
Rys. 17. Diagram aktywności funkcji sprawdzającej wystąpienie zakończenia gry..... 42
Rys. 18. Diagram aktywności funkcji sprawdzającej wystąpienia mata..... 43
Rys. 19. Diagram aktywności funkcji sprawdzającej wystąpienia pata..... 45
Rys. 20. Diagram aktywności funkcji sprawdzającej wystąpienie remisu przez .....
brak materiału..... 46
Rys. 21. Diagram aktywności funkcji sprawdzającej wystąpienie remisu przez .....
trzykrotne powtórzenie pozycji..... 47
Rys. 22. Diagram sekwencji cyklu życia klasy zarządzającej przebiegiem .....
gry szachowej..... 49
Rys. 23. Diagram sekwencji cyklu życia klasy zarządzającej przebiegiem turnieju .....
szachowego 52

Spis kodów

Kod. 1. Enumeracja ‘ChessPiece’ 30
Kod. 2. Klasa ‘ChessFigure’ 31
Kod. 3. Struktura ‘MoveInfo’ 31
Kod. 4. Funkcja zliczająca figury mogące się ruszyć na zadane pole 36
Kod. 5. Funkcje walidacji poprawności poruszania się wybranych figur..... 38
Kod. 6. Enum ‘DrawType’..... 39
Kod. 7. Struktura ‘NotationInfo’..... 40
Kod. 8. Funkcja sprawdzająca wystąpienie remisu z zasady pięćdziesięciu ruchów 44
Kod. 9. Funkcja ujednoznaczniająca pole poruszanej figury..... 48
Kod. 10. Funkcja obliczająca zmianę rankingu ELO gracza po zakończeniu partii..... 51
Kod. 11. Inicjalizacja silnika szachowego oraz kolejki odczytu jego odpowiedzi 55
Kod. 12. Przykład punktu końcowego ‘/go’ serwera z silnikiem szachowym 56

Załączniki

1. Płyta CD/DVD zawierająca:
    a. Prezentację wyników pracy dyplomowej
    b. Kody źródłowe oprogramowania